

TECHNISCHE UNIVERSITÄT DRESDEN

FACULTY OF ELECTRICAL ENGINEERING AND INFORMATION
TECHNOLOGY

CHAIR OF HIGHLY-PARALLEL VLSI SYSTEMS AND
NEURO-MICROELECTRONICS

VLSI Processor Design

Kreacher: A RISC-V based SoC Design



Submitted by group: prz25/group10
on March 16, 2026

Jhon Steven Pinto Hernandez
(5242341) [jhpi800g]

Luanjia Cheng
(5250916) [luch715g]

Sanjai Palanisamy
(5268661) [sapa892g]

Yongbo Wang
(5250403) [yowa870g]

Degree Program: M.Sc in Nanoelectronic Systems

Year of Matriculation: 2024

Supervisor: Dipl.-Ing. Stefan Scholze

Professor: Prof. Dr.-Ing. habil. Christian Mayr

Statement of Authorship

We hereby declare that we have written this thesis on the topic:

Kreacher: A RISC-V based SoC Design

entirely independently and have listed all used sources and aids. We are submitting this thesis for the first time as part of an examination. We understand that attempted deceit will result in the failing grade „not sufficient“ (5.0).

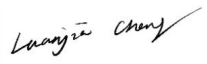
Last Name: Pinto Hernandez
First Name: Jhon Steven
Matriculation Number: 5242341

March 16, 2026
Date


Signature

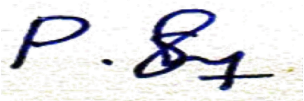
Last Name: Cheng
First Name: Luanjia
Matriculation Number: 5250916

March 16, 2026
Date


Signature

Last Name: Palanisamy
First Name: Sanjai
Matriculation Number: 5268661

March 16, 2026
Date


Signature

Last Name: Wang
First Name: Yongbo
Matriculation Number: 5250403

March 16, 2026
Date


Signature

Contents

1	Introduction	4
1.1	Project Overview	4
1.2	Design Objectives	4
2	RISC-V Core Design and Implementation	5
2.1	Single-cycle Core	5
2.1.1	Main Modules	6
2.1.1.1	IF_HK	6
2.1.1.2	ID	7
2.1.1.3	EX	8
2.1.1.4	MEM and WB	11
2.1.2	Core Interrupt Handling	12
2.2	Pipelined Core	13
2.2.1	Architecture Overview	13
2.2.1.1	IF	13
2.2.1.2	ID	14
2.2.1.3	EX	14
2.2.1.4	MEM and WB	16
2.2.2	Core Interrupt Handling	17
3	System-on-Chip (SoC) Components	18
3.1	Init Memory Controller	18
3.1.1	Init Controller	18
3.1.2	Memory Controller	20
3.2	PMEM interface top	20
3.2.1	PMEM Interface	21
3.2.2	Odd-Even Handler	21
3.2.3	Scheduler (Structural Hazard Resolution)	23
3.2.4	Program Memory (PRAM)	23
3.3	SPI master Interface	23
4	Functional Verification	26
4.1	Introduction to our verification framework	26
4.2	Software components of the verification framework	26
4.2.1	Random instruction generation tools	26
4.2.2	Riscv-unknown-elf-gcc cross-compiler	28

4.2.3	Verification_commits module	29
4.2.4	Interrupt Generator	30
4.2.5	Top Level Testbench	30
4.2.6	Spike simulator	32
4.2.7	Trace comparison python script	33
4.3	Coverage result	35
5	Synthesis and Physical Design	37
5.1	Synthesizability check of Kreachers SoC	37
5.2	Synthesis Results	37
5.2.1	Timing report analysis	37
5.2.2	Power report analysis	38
5.2.3	Area report analysis	39
5.3	Physical Design	40
5.3.1	Bind	40
5.3.2	Floor Planning and Power Planning	41
5.3.3	Placement	41
5.3.4	Clock Tree Synthesis	41
5.3.5	Routing	41
5.3.6	Signoff	42
5.4	Discussion and Conclusion	42
5.5	Future Work	43
6	Validation	44
7	Appendix	48

Distribution of Work

Name	Contribution
Jhon Steven Pinto Hernandez	• Team leader: Technical guidance through all modules and flow steps, project planning, task assignment and architectural key decisions. • Full Design and verification support of the RISC-V Single Cycle core. • Full Design and verification support of the RISC-V pipelined core. • Setup of Design environment in icpro (using SVN) and Vivado (using Git). • Physical implementation: Optimization for Placement & Routing.
Luanjia Cheng	• Design of the verification framework • Functional verification and debugging of the cores throughout the development • Validation of the cores after physical implementation • Software maintenance in icpro environment
Sanjai Palanisamy	• Design of the Init and memory controller modules • Design of PMEM interface module • Design and verification M-extension of RISC-V core • SoC integration • Physical design, Synthesis implementation and optimization
Yongbo Wang	• Design of the SPI module • HAL check and solution of warnings and errors • Design of C programs • Participated in parts of the C program verification work.

1 Introduction

1.1 Project Overview

The Kreacher project aims at the design and implementation of a RISC-V-based microcontroller integrated into a general-purpose system-on-chip (SoC). According to the task description, the system features a 64-bit little-endian datapath, a fully synchronous single-clock architecture, on-chip program memory, interrupt support, an external SPI interface, and program memory initialization after reset. The architecture of the Kreacher SoC includes a RISC-V processor core, 32 kB on-chip SRAM used as program memory (PRAM), an SPI master interface, an initialization controller, and a memory controller. In addition, the basic system supports two external interrupt inputs. Together, these modules form the complete SoC architecture specified in the task description. Therefore, the project covers not only the implementation of the processor core, but also the integration of the memory subsystem, SPI interface, initialization logic, and interrupt support. The final outcome of the work is an RTL-level Kreacher SoC design that is functionally verified in simulation using the provided software and testcases.

1.2 Design Objectives

The main objective of the Kreacher project is to realize a 64-bit little-endian RISC-V microcontroller within a fully synchronous single-clock SoC architecture. According to the task description, the processor core is based on the RV64IMC ISA with Zicsr support, while smaller project groups may omit the compressed instruction extension. In addition, selected privileged instructions and machine-level CSRs are required to support interrupt and trap handling in machine mode. A second objective is the implementation of a complete memory and communication subsystem around the processor core. The SoC includes 32 kB of on-chip SRAM as program memory (PRAM) and an SPI master interface for accesses to external memory or peripherals. The memory system must support the address mapping defined in the specification and allow both internal PRAM access and external bus transactions. In addition, after reset, the program memory has to be initialized through the external SPI interface before normal program execution starts. Another important objective is the support of basic interrupt functionality. In the basic system configuration, the SoC provides two external interrupt inputs, which are handled at instruction boundaries. IRQ0 has higher priority than IRQ1 if both are pending, and the interrupt mechanism relies on the machine-status interrupt-enable bit and dedicated acknowledge signals. This requires the design to implement the specified handler addresses and the corresponding control behavior for interrupt entry and return. Since the project is carried out at RTL and simulation level, the final design objective is not a fabricated chip or physical hardware prototype, but a functionally correct SoC implementation that can be verified in simulation. For this purpose, the provided GCC-based software environment, linker setup, default handlers, and testcase flow are used to validate the implemented hardware functions with assembly and C programs.

2 RISC-V Core Design and Implementation

This chapter presents the design of the Kreachter RISC-V processor cores. Since the project includes both a single-cycle design and a pipelined design, the two cores are described separately in the following sections. For each core, the implementation of the main architectural components is introduced, including the overall structure, instruction fetch and PC logic, decoder, ALU, register file and CSRs, and interrupt handling. This organization makes the description clearer and helps distinguish the implementation features of the two core variants.

2.1 Single-cycle Core

The first processor variant developed in this project is a single-cycle RISC-V core. In this design, the major functions required for instruction execution are organized into a single-cycle datapath together with the corresponding control logic. Although the core is functionally divided into several regions, including instruction fetch, decode, execution, memory-related processing, and write-back, it remains a single-cycle implementation rather than a deeply pipelined processor. The purpose of this section is to describe how the single-cycle core is structured internally and how its main architectural components are realized.

The high-level architecture of the single-cycle core is illustrated in Fig. 2.1. The core's main datapath is organized into five main functional modules, namely IF_HK, ID, EX, MEM, and WB, which correspond to instruction fetch and housekeeping, instruction decode, execution, memory-related processing, and write-back, respectively. In addition, a global Pause Handler spans the entire architecture and coordinates pause- and sleep/wakeup-related control behavior.

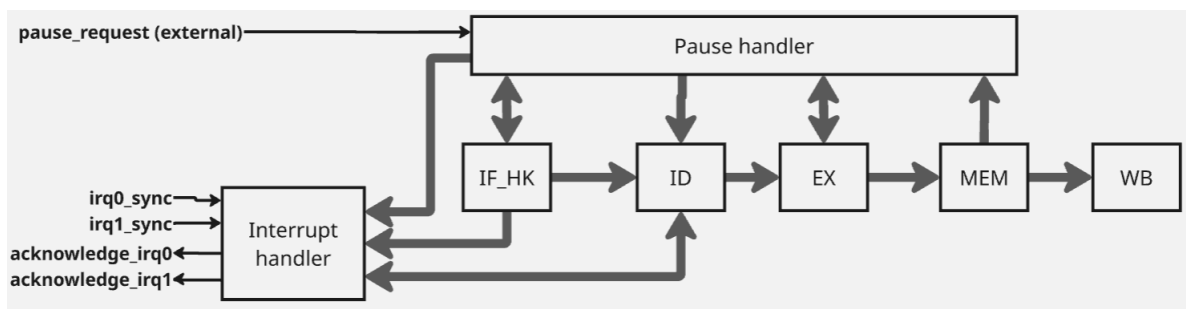


Figure 2.1: Single Cycle Core top block diagram.

2.1.1 Main Modules

2.1.1.1 IF_HK

The IF_HK module (Fig. 2.2) contains the fetch-side logic, including the housekeeping block, fetch block, and mini controller. Based on the signal organization shown in the figure, the IF_HK module is responsible for program-counter-related control, instruction fetching, and auxiliary fetch-side control operations.

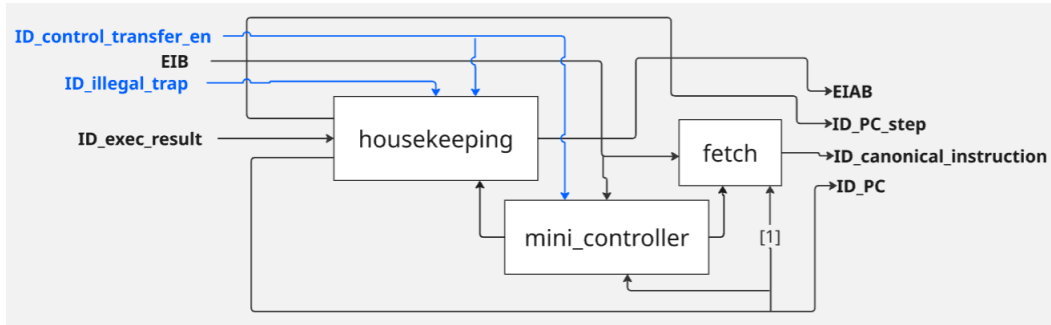


Figure 2.2: IF_HK module block diagram.

The Housekeeping block contains the PC register and the logic for selecting the next PC value. Depending on the control signals `control_transfer_en` and `illegal_trap`, the PC is updated either by normal sequential increment or by an alternative control-transfer result. Dedicated +2 and +4 increment paths, indicate that different PC step sizes are supported depending on the fetched instruction format (either compressed or canonical instructions).

The Fetch block uses the current PC as the instruction address to access the instruction bus (EIB). It's main purpose is to convert the respective EIB fetches into a canonical instruction, as well as to save any incomplete RV64I instructions in a register `EIB_2_temp`, in order to enable the core to fetch the next memory row, concatenate and decode within the next cycle after an incomplete RV64I has been detected. This logic is illustrated in Fig. 2.3.

Cycle	EIB		Operation
	EIB_2	EIB_1	
0	RVI_1	C	Save EIB_2 into EIB_2_temp
1	C	RVI_2	Concatenate EIB_1 with EIB_2_temp

Figure 2.3: Save-and-Concatenate algorithm used in Fetch block.

The previously described algorithm is controlled by the mini_controller block. Its functionality resides in taking the quadrants from the fetched halfwords (EIB_1, EIB_2) as well as the current fetch pointer (PC[1]), and generates the necessary control signals needed in the housekeeping and fetch blocks to control when to pre-fetch the next row, save and concatenate.

In summary, the IF_HK module serves as the front-end of the single-cycle core by combining PC management, instruction fetch, and instruction preparation in one single region.

2.1.1.2 ID

At the output of the IF_HK module, the fetched instruction is then transferred into the ID module, which contains the Control, Regfile, build imm, and CSR bank blocks. This indicates that instruction decoding, source operand readout, immediate generation, and CSR-related operations are mainly handled within this single module (Fig. 2.4).

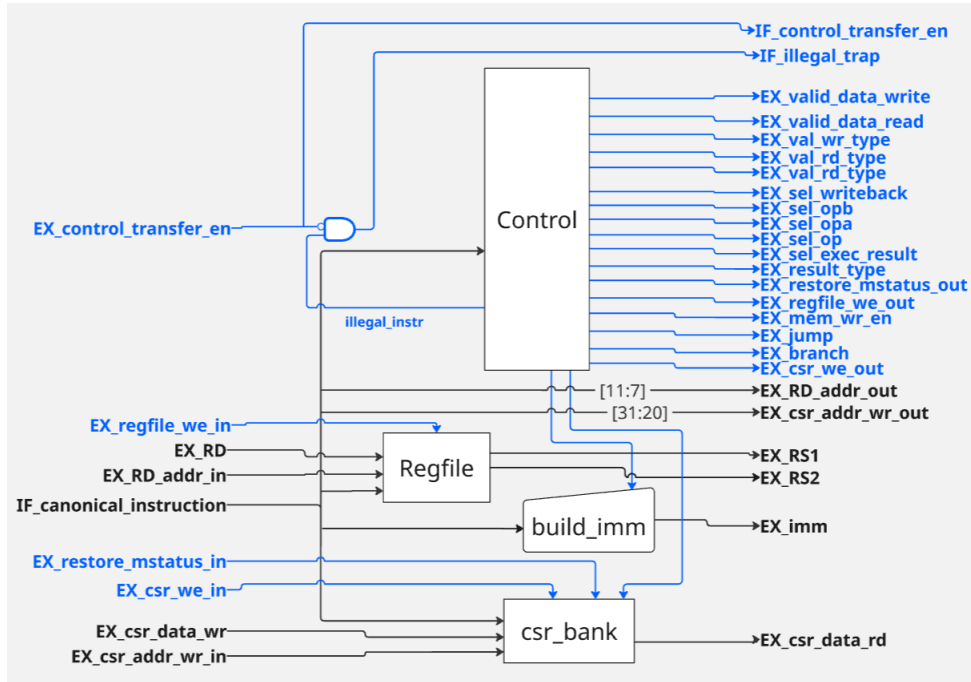


Figure 2.4: ID module block diagram

The instruction decoder of the single-cycle core is implemented in the ID stage and is centered around the control block. This is a fully combinational block and its main function is to interpret the canonical_instruction generated by the front-end and translate the encoded instruction fields into the control signals required by the later datapath stages. Therefore, it acts as the central control source of the processor and determines how each instruction is executed, whether it affects control flow, and which datapath resources are activated.

Internally, the control block extracts the key instruction fields directly from the 32-bit canonical instruction. These fields are used to distinguish arithmetic, memory, branch, jump, privileged, and CSR-related instructions. In addition, the decoder generates an illegal_instr signal whenever an unsupported opcode or illegal operation form is detected. Internally, the control block explicitly distinguishes different illegal cases, including invalid opcode, invalid ALU operation, invalid memory access type, invalid branch operation, invalid CSR operation, illegal CSR read, and illegal CSR write.

Immediate generation is separated from the main decode logic through the imm_type signal. This is an

input of the `build_imm` block, which selects among I-type, S-type, B-type, U-type, J-type, and Zicsr immediate formats, as well as an `IMM_NOT_USED` case in order to produce the respective immediate of an instruction format. This allows the datapath to construct the correct operand format for arithmetic instructions, memory instructions, control-transfer instructions, and CSR-immediate operations without overloading the main control logic.

Since the core also implements the C extension, the overall decode path must support variable-length instructions in addition to the standard 32-bit format. In the current architecture, this is handled before the main decoder. Hence, the ID stage always receives a `canonical_instruction`, which means that compressed and uncompressed instructions are normalized into a common representation in the front-end before entering the `control` module. Moreover, the single-cycle core implements its architectural state through two main storage structures: the general-purpose register file and the machine-level control and status registers (CSRs). In the design, these two parts are implemented separately as the `regfile` and `csr_bank` modules (Figure. 2.4).

2.1.1.3 EX

The EX region contains operand-selection multiplexers, the ALU, and the PC+imm logic. These blocks perform arithmetic and logical operations, branch-related evaluation, and PC-relative address generation. It's main control signals are generated in the decode region and then propagated to the subsequent functional regions.

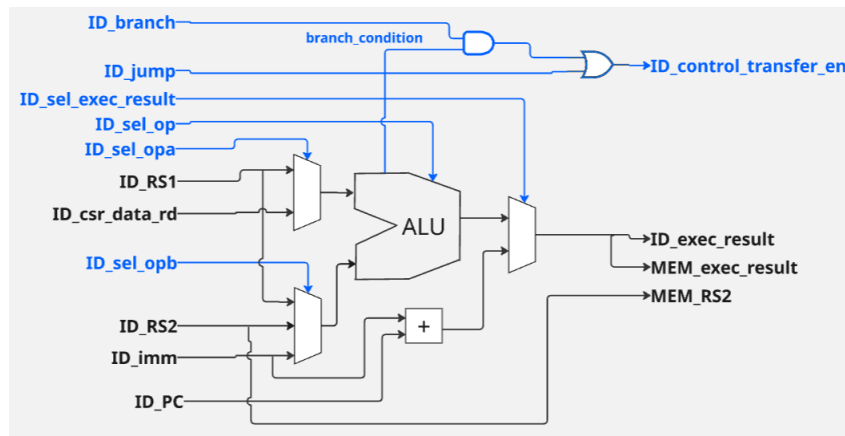


Figure 2.5: EX module block diagram

The ALU is the main computational block of the EX stage. It receives the selected operands together with a 6-bit operation selector `sel_operation` signal, and produces both a numerical result `ALU_result` and a separate branch decision output `branch_condition`. The ALU directly implements forwarding, arithmetic, logical operations, signed and unsigned comparisons, shift operations, CSR-related bit manipulation, and M-extension operations. In this way, a unified execution unit is reused across arithmetic instructions, memory address generation, branch instructions, jump-related instructions, CSR operations, multiplication, and division.

Besides the ALU path, the EX stage also contains a dedicated `PC_plus_imm` adder. This path is used for instructions whose execution result is PC-relative rather than a direct ALU output. Branch and jump handling are integrated directly into the execution stage. Unconditional jumps are enabled directly by decode control, while branch redirection depends on the runtime comparison result produced by the ALU. In this way, the EX stage combines instruction execution and control-flow resolution in a single datapath stage.

Support for the M extension is implemented through two iterative submodules, `multiplier_top`, and `divider_top`. The multiplier uses a radix-4 Booth multiplication algorithm in `radix_4_booth_multiplier` and generates a double-width product, from which the ALU selects the lower XLEN bits, upper XLEN bits, or sign-extended lower 32 bits depending on the operation (`MUL`, `MULH`, `MULHU`, `MULHSU`, `MULW`).

The Figure.2.6 describes the FSM state of the `radix_4_booth_multiplier`,

S_IDLE It is the waiting state. It waits here until the `start` input goes high. Once `start` is detected, it transitions to `S_LOAD`.

S_LOAD It is the setup state. It only stays in this state for one clock cycle, and it moves to `S_ITERATE`.

S_ITERATE As this is a Radix-4 multiplier, it processes 2 bits of the multiplier per cycle. For a 64-bit input (`XLEN = 64`), it requires 32 cycles. The FSM checks if `iteration_counter >= MAX_ITERATIONS` (which is set to 32, and decremented in each cycle). If false, it loops back to `S_ITERATE`. If true, it moves to `S_DONE`.

S_DONE It asserts `done = 1`, signaling to the external system that the `prod_result` contains the valid 128-bit product. It moves to `S_IDLE` on the next clock edge to await the next operation.

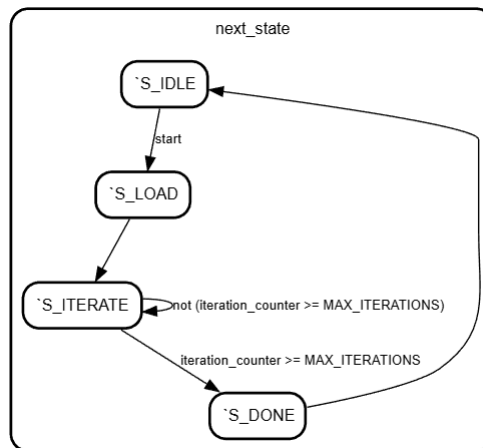


Figure 2.6: FSM of Radix-4 Booth Multiplier.

The divider uses a radix-4 division algorithm in `radix_4_divider` and produces both quotient and remainder outputs for `DIV`, `DIVU`, `REM`, `REMU`, and their W-type variants. Since these operations are iterative, the ALU exports `pause_to_calculate`, and the EX stage forwards it as `EX_pause_request`, allowing the core to stall until multiplication or division completes.

The Figure.2.7 illustrates the FSM state of the `radix_4.booth.multiplier`,

S_IDLE It is the waiting state. It waits for the `start` signal, and moves to the next state when `start` is high.

S_LOAD The divider raises its `busy` flag. It spends exactly one clock cycle to latch values of the `dividend` and `divisor` into the registers and sets the counter value.

S_ITERATE Since it is a Radix-4 divider, it takes 2 bits of the dividend per clock cycle, comparing the value and updating the remainder and quotient registers. The counter ticks down by 2 each cycle. Once the counter hits 2 or less, the division is complete.

S_DONE The divider asserts the `done` flag, signalling that the final quotient and remainder are ready. And jumps to the `S_IDLE` state in the next clock cycle.

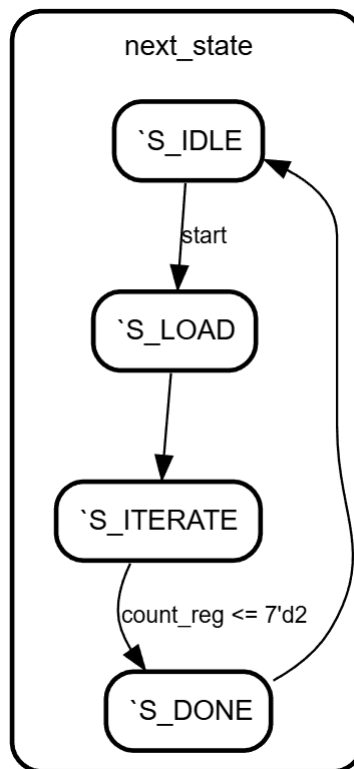


Figure 2.7: FSM of Radix-4 Divider.

A particularly important part of the execution datapath is the implementation of signed division. In `divider_top`, the design first determines whether the current operation is signed or unsigned. For signed operations (`DIV`, `REM`, `DIVW`, `REMW`), both operands are treated as signed; for unsigned operations (`DIVU`, `REMU`, `DIVUW`, `REMUW`), both are treated as unsigned. For W-type instructions, the operands are first reduced to their lower 32 bits and sign-extended. The divider then computes the sign flags of the dividend and divisor and derives their absolute values, `dividend_abs` and `divisor_abs`. These absolute values are used both for quick-result detection and for the iterative radix-4 division datapath itself.

Inside `radix_4_divider`, the actual division algorithm operates on the absolute values of the operands rather than on the original signed values. The iterative quotient and remainder magnitudes are computed from `dividend_abs` and `divisor_abs`, exactly as in an unsigned magnitude-division flow. After the iterative process finishes, the signs are restored: the quotient sign is determined by the XOR of the operand signs, while the remainder sign follows the sign of the dividend. Consequently, signed division is not implemented as a completely separate algorithm from unsigned division. Instead, the design reuses the same magnitude-based division core and adds sign preprocessing and result-sign restoration around it. This directly satisfies the requirement that signed division should follow the same absolute-value computation principle as unsigned operations.

The divider also implements several fast special cases before starting the iterative algorithm. These include division by zero, zero dividend, divisor magnitude larger than dividend magnitude, and equal-magnitude division. In such cases, the quotient and remainder are generated immediately without invoking the radix-4 divider. For example, division by zero produces an all-ones quotient and returns the dividend as the remainder, while equal-magnitude division produces $+1$ or -1 depending on the operand signs. Only when these fast cases do not apply does the iterative divider start and request the core to pause.

Overall, the EX stage and ALU together form a structured execution datapath for the single-cycle core. Operand selection, ALU processing, PC-relative result generation, branch resolution, multiplication, division, and result selection are all integrated within the same stage. In particular, the divider implementation demonstrates a clear datapath principle: convert operands to absolute values, perform magnitude division using the same core algorithm as in the unsigned case, and then restore the architectural sign of the result.

2.1.1.4 MEM and WB

The MEM module (Fig. 2.8) includes the extend result, extend read, and extend write blocks. Together with the external memory bus interface (EMCB, EMAB, EMDb), this region is responsible for memory-related data formatting and interface handling. Finally, the WB region (Fig. 2.9) contains the write-back selection logic, where the final result source is selected and forwarded to the register write-back path.

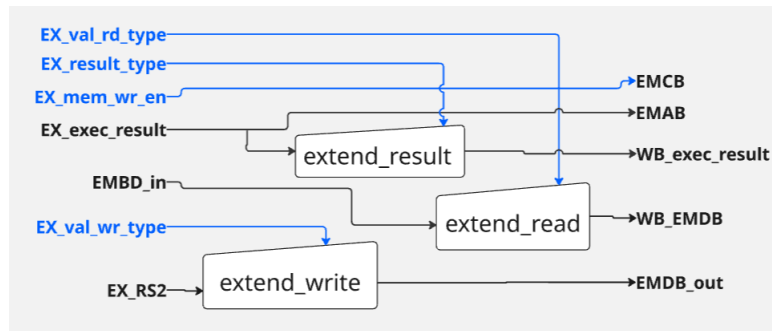


Figure 2.8: MEM module block diagram

Within the MEM module, the extend result unit is used for sign extension of the result from the EX stage when it is required by an instruction. The extension read unit is used specifically when the core executes an load instruction. It performs either sign extension or zero extension depending on the type

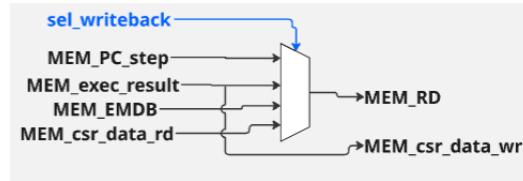


Figure 2.9: WB module block diagram

of load word instruction. The external memory bus always reads 64 bits at a time, and in order to apply masks to the valid part of the data from the bus, the module reads the second and third bits from the ALU where the target address of the load word instruction is calculated and apply masks based on the two bits. The extend write unit is similar to the extend read unit, but designed for store instructions. It performs sign/zero extension and apply masks to the data that will be written to a memory.

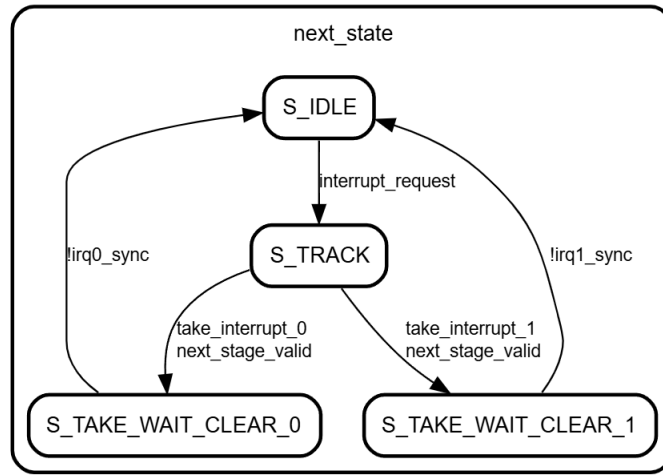


Figure 2.10: FSM of interrupt handler

2.1.2 Core Interrupt Handling

The interrupt and trap handling mechanism of the single-cycle core is implemented through the cooperation of the interrupt handler, decode-stage control logic, the CSR bank, and the pause-control logic. Together, these components determine when traps are taken, how trap-related state is recorded, and how control flow is redirected during trap entry and return.

The design supports external interrupts and illegal-instruction exceptions. External interrupts are detected and coordinated by the interrupt handler, while illegal instructions are identified during instruction decoding. Before an interrupt is taken, the processor ensures that the current instruction has reached a safe architectural boundary so that program execution can later resume correctly.

When a trap occurs, the processor records the cause of the trap and the address to which execution should return after the handler finishes. These updates are managed by the CSR subsystem, which maintains the machine-level trap state and ensures that interrupts are temporarily disabled while the trap handler executes.

In addition to trap entry, the processor also supports trap return and interrupt-driven wake-up. Returning from a trap restores the previous interrupt state and resumes execution at the recorded return address. The

processor can also enter a sleep state while waiting for an interrupt, during which execution is paused and automatically resumed once an interrupt becomes active.

2.2 Pipelined Core

The second processor implementation developed in this project is a pipelined RISC-V core. In contrast to the single-cycle design, this version organizes instruction execution into multiple pipeline stages and allows different instructions to occupy different stages at the same time. As a result, the datapath and control logic must not only perform instruction execution, but also manage inter-stage data transfer, hazard handling, and control-flow recovery.

The high-level architecture of the pipelined core is shown in Fig.2.11. The processor is organized into five pipeline stages, namely IF_HK, ID, EX, MEM, and WB, corresponding to instruction fetch and house-keeping, instruction decode, execution, memory access, and write-back. Unlike the single-cycle design, these stages are separated by explicit inter-stage registers, so that different instructions can progress through the datapath in an overlapped manner. In addition, the core includes a global Pause Handler and a dedicated Hazard Control Unit, both of which are essential for maintaining correct execution in the presence of data hazards, control hazards, and stage stalls.

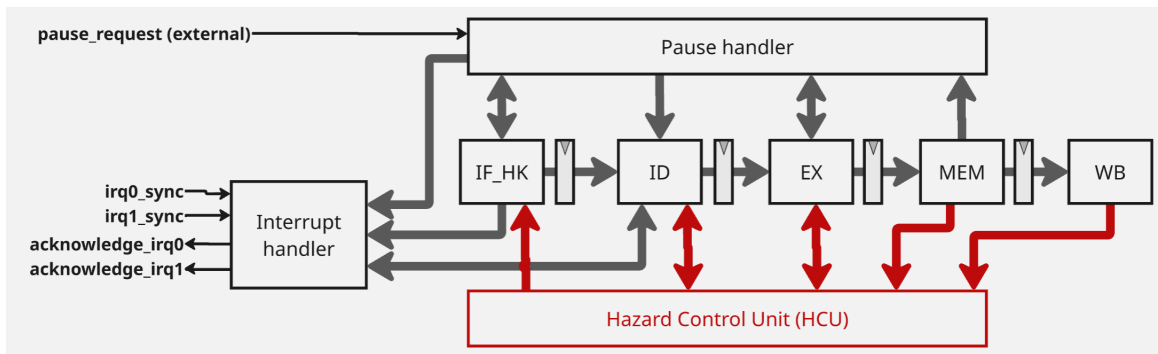


Figure 2.11: Pipelined Core top block diagram.

A major architectural difference from the single-cycle core is the explicit handling of pipeline hazards. As indicated in Fig. 2.11, the Hazard Control Unit monitors dependencies across multiple stages and generates control signals such as stall and flush. These signals interact with the IF_HK, ID, and EX stages to prevent incorrect execution when register dependencies or control transfers occur. At the same time, write-back-related and forwarding-related information is propagated across the pipeline so that later instructions can be coordinated correctly with earlier ones.

2.2.1 Architecture Overview

2.2.1.1 IF

In the pipelined core, the instruction fetch and PC update logic remains functionally similar to the single-cycle version, but its control behavior becomes significantly more complex because the front-end must now interact with stage registers, hazard handling, and pipeline recovery. As shown in Fig. 2.12, the IF_HK stage is composed of the same three main blocks as before, namely `mini_controller`,

fetch, and housekeeping, but their outputs are now coupled to the Hazard Control Unit (HCU) and to the explicit IF/ID stage register.

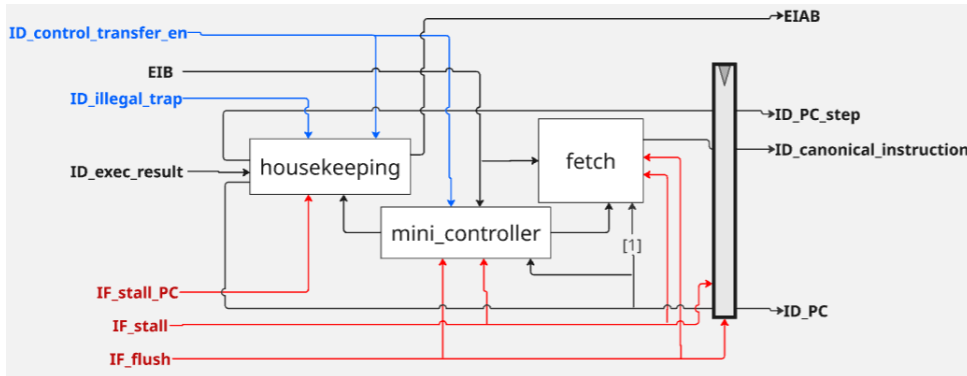


Figure 2.12: IF_HK module block diagram

The main difference in the pipelined version is that the actual PC update must now respect hazard and pause conditions. In housekeeping, the internal `next_PC` is not always equal to `next_program_PC`. If `pause_to_concatenate`, `EX_pause_request`, or the HCU-generated `stall` signal is active, the PC is held constant. Otherwise, interrupt acknowledgments have priority, and the PC is redirected to `PC_IRQ0` or `PC_IRQ1` when `acknowledge_irq0` or `acknowledge_irq1` is asserted. Only when no stall or interrupt redirection is present does the front-end advance to `next_program_PC`.

2.2.1.2 ID

In the pipelined core, the main instruction-decoding functionality is largely reused from the single-cycle design, since the same `control` module is used to decode the canonical 32-bit instruction and to generate the corresponding control signals. Therefore, the fundamental decoding rules for RV64IMC, Zicsr, and the compressed-instruction front-end representation remain unchanged. The main architectural difference lies not in the decoding table itself, but in how the decode results are incorporated into the pipeline and how they interact with hazard-control logic.

This stage-boundary behavior is controlled by explicit flush logic. When `ID_flush` is asserted, this clear operation is applied instead of normal stage transfer. Architecturally, this corresponds to inserting a bubble into the pipeline. As a result, wrong-path instructions or instructions invalidated by hazard handling do not propagate into the execution stage with stale control signals.

2.2.1.3 EX

The EX stage of the pipelined core is built around three main elements: operand-bypass logic, the ALU execution path, and the EX/MEM stage register boundary. At the input side, the execution stage does not directly consume the values coming from the ID stage without modification. Instead, it first applies bypass selection under the control of the Hazard Control Unit. In the RTL, `RS1` and `RS2` can be selected

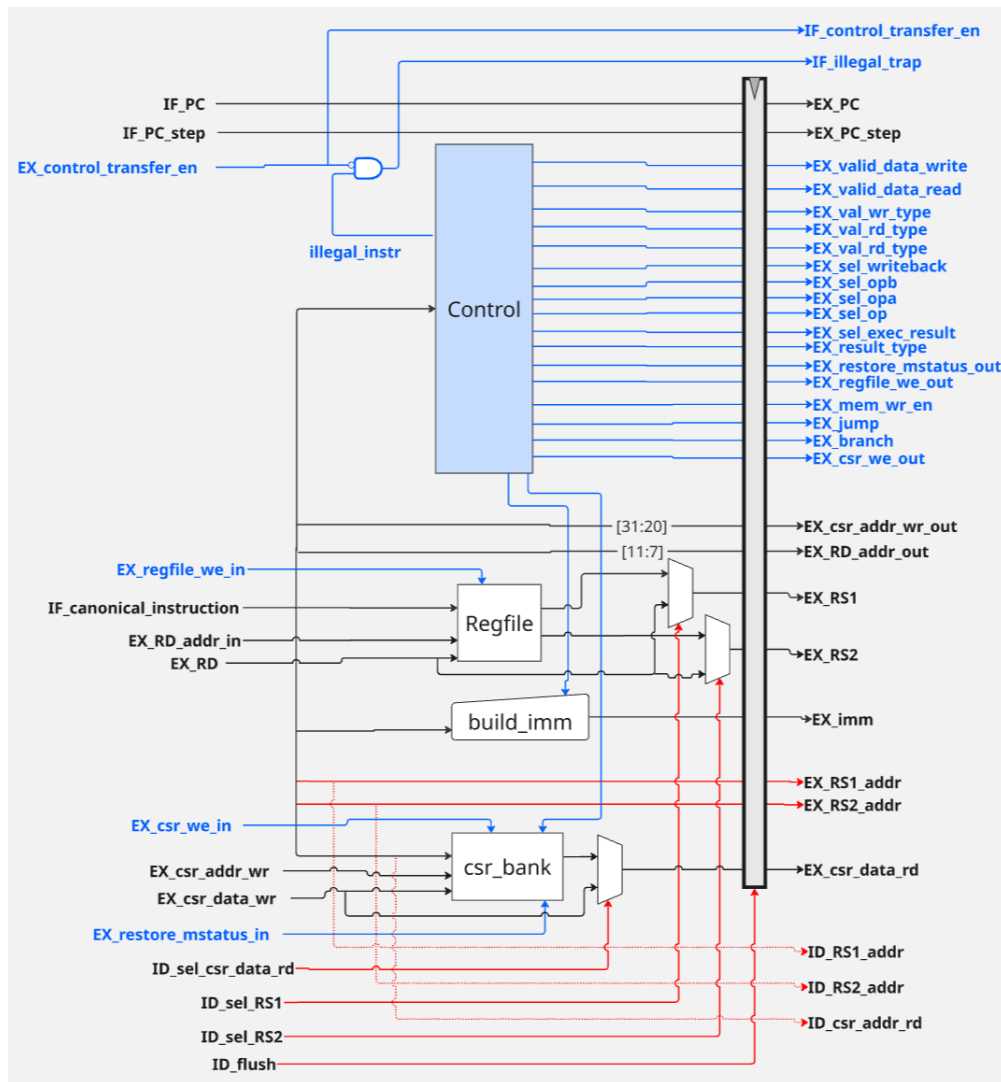


Figure 2.13: ID module block diagram

either from the normal ID-stage operand values, from the MEM-stage forwarded execution result, or from the WB-side write-back data. Similarly, CSR read data can also be bypassed either from the MEM execution path or from the CSR write-back path. This means that, in the pipelined core, the EX stage is the main point at which recent producer results are forwarded to dependent instructions.

A further pipeline-specific addition is the `csr_mask` block. In the EX stage, the CSR read value used for ALU operations is not always forwarded directly. Instead, it is first passed through `csr_mask`, which enforces the writable-field restrictions of machine CSRs such as `mstatus`, `mepc`, and `mcause`. This prevents invalid updates to read-only or reserved CSR bits before the value is used in a CSR-modifying instruction. Such masking is not a generic ALU concern in the single-cycle description, but becomes an important execution-stage detail in the pipelined implementation because CSR updates may interact with forwarding and write-back timing.

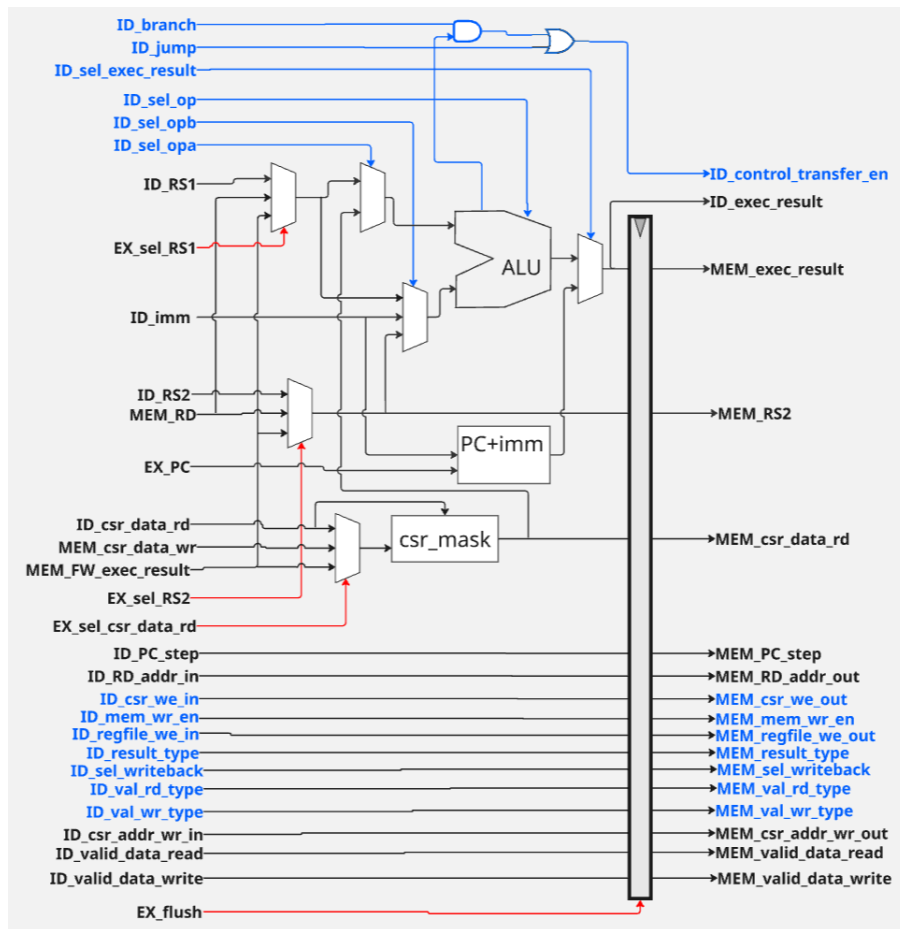


Figure 2.14: EX module block diagram

2.2.1.4 MEM and WB

The MEM module of the pipelined core shares the same functionality as the one in the single cycle core. The only difference resides on the signals that pass through the stage register. On the other side, the WB stage is exactly the same as the one in Fig. 2.9.

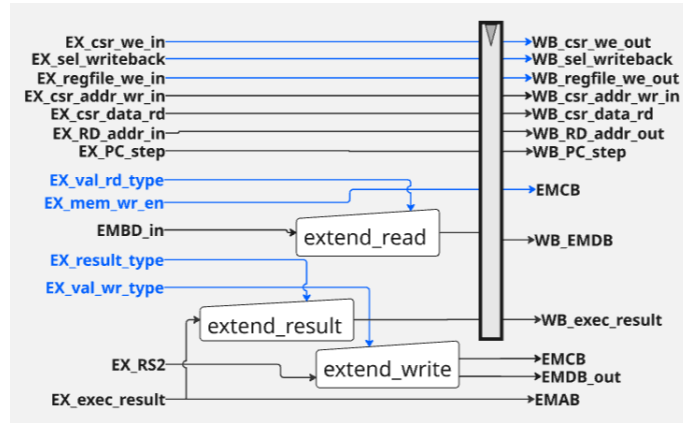


Figure 2.15: MEM module block diagram

2.2.2 Core Interrupt Handling

Interrupt and trap handling in the pipelined core differs from the single-cycle implementation because several instructions may already be in flight when an interrupt occurs. Therefore, the processor must determine a precise interrupt-take point and ensure that only architecturally valid instructions affect the machine state. This functionality is implemented through the cooperation of the interrupt handler, the Hazard Control Unit (HCU), the CSR bank, and the pause-control logic. An important clarification here is that the FSM used for the interrupt handler is the same as in Fig.2.10.

When an interrupt becomes pending, the interrupt handler tracks the progress of the currently executing instruction and delays trap entry until a safe architectural boundary is reached. This mechanism ensures that the correct return address is preserved and that program execution can resume correctly after returning from the trap.

Once an interrupt is accepted, the Hazard Control Unit performs pipeline recovery by flushing younger instructions from the pipeline. This establishes a precise interrupt boundary and prevents partially executed or speculative instructions from affecting the architectural state. Illegal-instruction traps are handled within the same control framework.

The `WFI` instruction interacts with the pipeline through the pause-control logic. When sleep mode is entered, pipeline execution is suspended and automatically resumed when an interrupt becomes active. Together, interrupt tracking, pipeline flushing, and pause control ensure correct trap handling in the presence of multiple active pipeline stages.

3 System-on-Chip (SoC) Components

This chapter details the design and implementation of the key modules that make up the Kreacher SoC. It outlines the design process and highlights the specific technical challenges encountered during implementation. As illustrated in Figure 3.1, the Kreacher SoC is built around four major components: the five-stage pipelined RISC-V core, the initialisation memory controller (Init memory controller), the program memory (PMEM) interface, and the Serial Peripheral Interface (SPI) module.

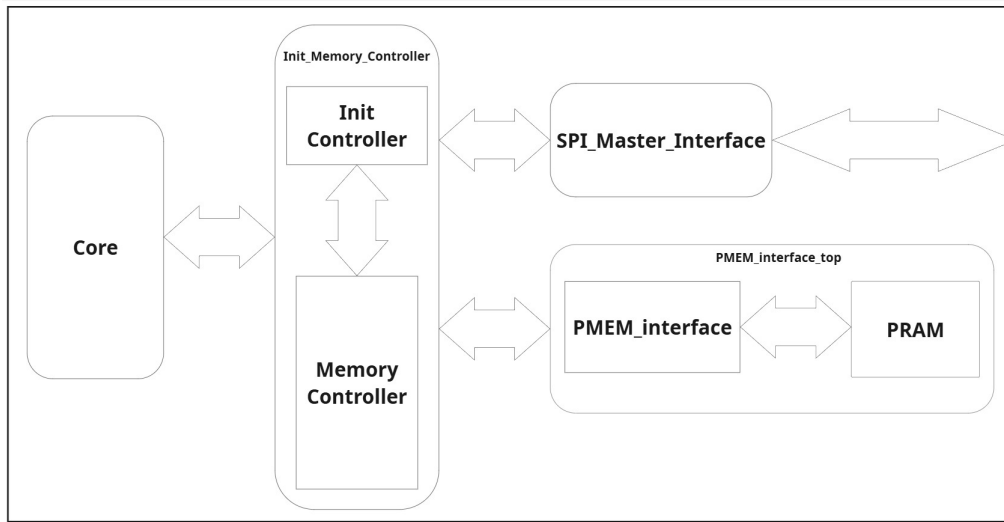


Figure 3.1: Block diagram of the Kreacher SoC.

3.1 Init Memory Controller

The Init Memory Controller acts as the central interconnect between the RISC-V core, the program memory interface, and the external data memory via the SPI protocol. The top-level module is logically divided into two submodules: the Init Controller and the Memory Controller. Its primary function is to route core addresses to their correct target destinations while seamlessly managing the internal 18-bit address mapping.

3.1.1 Init Controller

The Init Controller module acts as a Finite State Machine (FSM) responsible for controlling the initial boot sequence of the SoC, as well as handling load, store and custom masked store instructions. Its primary function upon reset is to initialize the 32 kB internal program memory by fetching instructions via the external SPI interface from external memory from addresses 0x0000 to 0x7fff. To achieve this, the controller utilizes an internal up-counter to generate memory addresses sequentially. The initialization finishes on the last address or alternatively if IRQ0 is activated. Once initialization is complete,

the FSM transitions into its normal operation phase, where it actively monitors and executes load, store, multi-cycle masked store instructions. Figure 3.3 shows the different FSM states of the Init Controller, which can be divided into two main phases:

Phase 1: Boot Sequence and Initialization

S_IDLE The reset state. The FSM remains here asynchronously during a system reset (`reset_n` is low). Once reset is released, it immediately transitions to start the initialization sequence.

S_START The first active cycle of the initialization phase. The FSM prepares the datapath to route data from the SPI interface to the PRAM using the internal address counter.

S_WAIT The active burst-read state via SPI. The controller waits while memory is populated sequentially. It remains in this state until the address counter reaches the memory limit (`0x7fff`) or the sequence is aborted by an active interrupt request.

S_FIRST_FETCH Triggered after the memory is successfully initialized or aborted. The FSM pre-fetches the first instruction for the RISC-V core before handing over bus control.

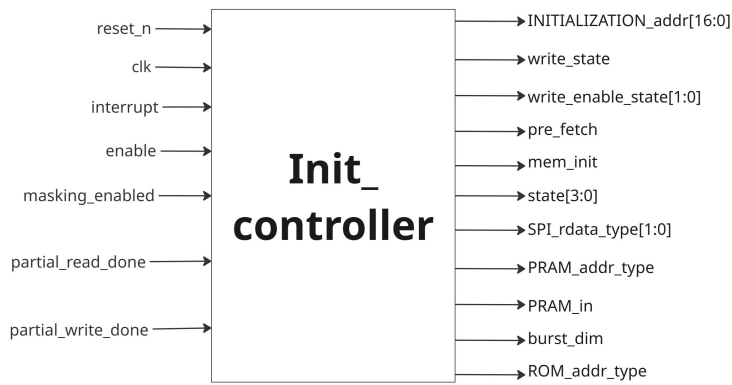


Figure 3.2: Block diagram of the Init Controller.

Figure 3.3: FSM of the Init Controller.

Phase 2: Normal Operation and Masked Store Execution

S_NORMAL_OP The default idle state during standard processor execution. The FSM monitors the `masking_enabled` signal. If a masked store instruction is detected, it triggers the custom modification sequence.

S_PARTIAL_READ The first step of the masked store. The FSM initiates a read of the existing data from memory and waits until the `partial_read_done` flag asserts that the data is ready.

S_APPLY_MASK A single-cycle transitional state where the memory controller datapath applies the bitmask to combine the newly written data with the existing data retrieved during the partial read.

S_PARTIAL_WRITE The FSM issues the write command to store the newly masked data back into memory, waiting for the `partial_write_done` signal to confirm the transaction is complete.

S_PARTIAL_STORE_DONE The final state of the partial store instruction. The FSM triggers a pre-fetch to keep the processor's pipeline running smoothly, and then returns to `S_NORMAL_OP` to await the next instruction.

3.1.2 Memory Controller

The Memory Controller acts as the central data router for the system, connecting the pipelined RISC-V core, the Init Controller, the SPI master interface, and the 32kB internal program memory (PRAM). It utilises the control signals generated by the Init Controller's FSM as select lines for internal multiplexers, dynamically routing data and addresses between these modules. The controller manages system memory accesses across four distinct operational phases:

- **Initialization Phase:** Immediately following a reset, the controller gives bus priority to the Init Controller. It routes instructions fetched from the external SPI interface directly into the internal PRAM. During this time, the RISC-V core is intentionally stalled using the `pause_request_initialization` signal to prevent it from executing invalid data.
- **Instruction Fetching:** Once initialization is complete, the controller transitions to its primary duty of fetching instructions for the core. It immediately forces a fetch from the reset handler at address `0x0008` to begin execution. During normal operation, it routes the core's instruction address (EIAB) to the PRAM and returns the 32-bit instructions (EIB).
- **Normal Load/Store Operations:** For data memory accesses, the controller evaluates the 64-bit address provided by the core to determine the correct destination. Based on the memory address, the controller recognizes this as an external memory address and routes the request to the SPI interface or to the `PMEM_interface_top`. Because SPI transfers require multiple clock cycles, the controller asserts the `pause_request_load_store` signal to safely stall the core until the external read or write finishes.
- **External Masked Stores (Read-Modify-Write):** The internal PRAM natively supports masked stores via the bitmask signal `BM_I`, but the external memory accessed via SPI does not. To bypass this limitation, the Memory Controller performs a read-modify-write sequence. It asserts `pause_request_partial_store` to stall the core, reads the existing 64-bit word from external memory, applies the core's bitmask (`EMCB_mask`) to merge the old and new data (`masked_wdata`), and writes the final word back over the SPI bus.

3.2 PMEM interface top

The PMEM Interface Top module manages all interactions with the on-chip internal Program Memory (PRAM). To achieve the required 32KB total capacity, the memory is physically constructed using 8 discrete SRAM macros of type `HM_1P_GF28SLP_1024x32_1cr`, each providing $1k \times 32$ bits (4KB) of storage. Although intended primarily to store executable code, the memory controller and core fully support write accesses to these macros during normal operation. To handle different data widths seamlessly, these 8 macros are organized into a 4×2 interleaved architecture. They are divided into 4 independent rows (handlers), with 2 macros (an "even" and an "odd" macro) per row. Because RISC-V instructions

are 32-bit aligned while the system utilizes a 64-bit data path, this arrangement is highly efficient: a 32-bit instruction fetch targets a single macro, whereas a 64-bit data load or store can span across both the even and odd macros simultaneously.

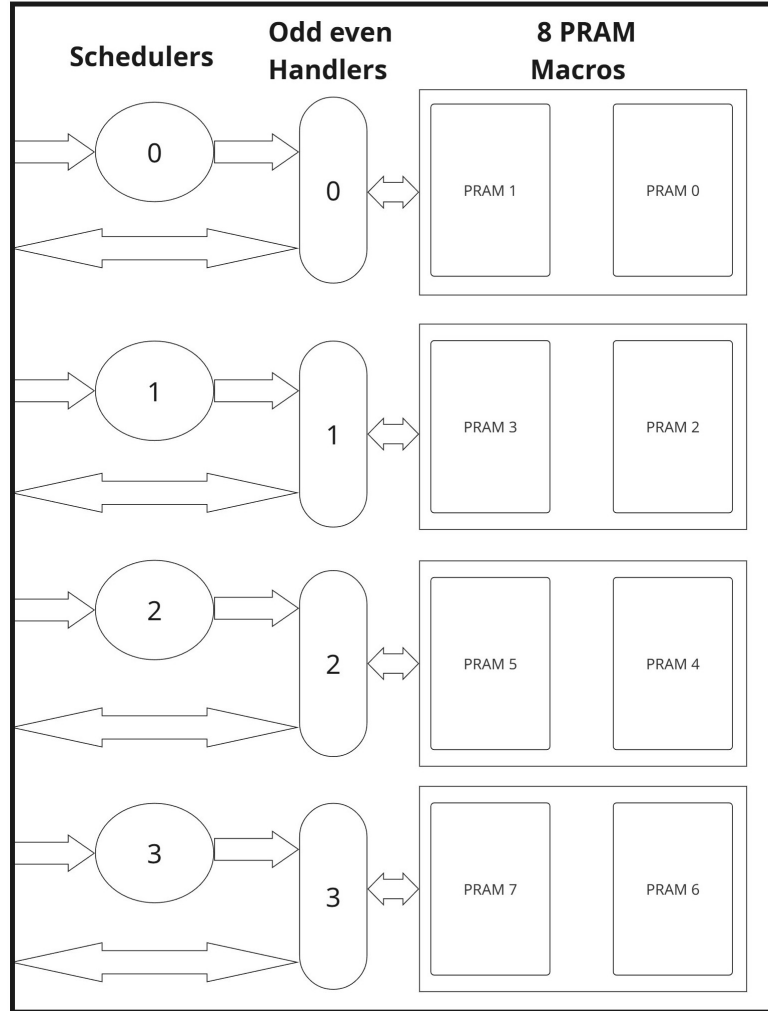


Figure 3.4: Block diagram of the PMEM interface top module.

The module is structurally divided into four distinct sub-components:

3.2.1 PMEM Interface

This core sub-module acts as the primary router and decoder. It interprets the incoming read/write addresses from the Memory Controller, determines which of the four memory handlers is being targeted, and generates the appropriate chip select (*cs*) and write enable (*we*) signals for the memory macros.

3.2.2 Odd-Even Handler

The Odd-Even Handler manages the physical data splitting required for 64-bit memory accesses. It translates incoming requests, dynamically routing the lower and upper 32 bits of a 64-bit data word to the "even" and "odd" SRAM macros, respectively. It also applies the correct bitmasks (*init_internal_mask*) to each macro independently during partial write operations.

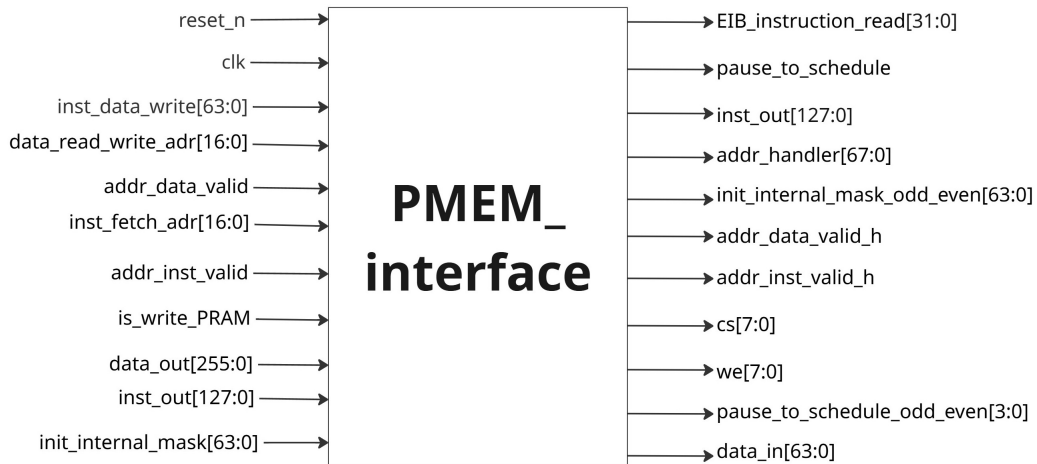


Figure 3.5: Block diagram of the PMEM interface module.

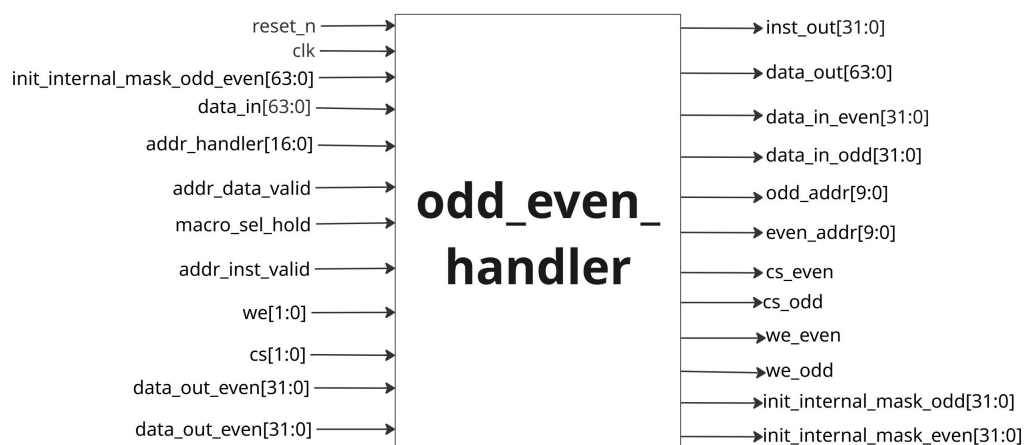


Figure 3.6: Block diagram of the Odd-Even handler module.

3.2.3 Scheduler (Structural Hazard Resolution)

In a 5-stage pipelined architecture, a structural hazard occurs if the core attempts an Instruction Fetch and a Data Load/Store to the same memory macro in the same clock cycle. The Scheduler module actively monitors the address buses to detect this collision (`cs_odd` and `cs_even`). When a conflict arises, the Scheduler resolves it by prioritising the data access and then the instruction fetch. It asserts the `pause_to_schedule` signal to stall the core for a single cycle, safely buffers the pending instruction fetch address in a holding register (`holded_addr`), and automatically executes the instruction fetch on the subsequent clock cycle once the data operation is complete.

3.2.4 Program Memory (PRAM)

This layer contains the physical instantiations of the 8 `HM_1P_GF28SLP_1024x32_1cr` SRAM macros. The macros receive the decoded addresses, chip selects, and masked data from the Odd-Even Handlers to perform the actual synchronous read and write operations.

3.3 SPI master Interface

The SPI Master Interface is responsible for handling external communication of the SoC, specifically executing external memory accesses and memory-like access using byte addresses from the system and 64-bit data words. While it accepts standard byte addresses from the Memory Controller, the interface achieves 64-bit alignment by internally not considering the three lowest address bits and converting them into a word address during transmission. The module implements a precise sequential protocol. Every transfer begins with a 16-bit header sent from the master to the slave. The very first bit specifies the operation type: 1 indicates write, and 0 indicates read. This is immediately followed by 14 bits byte address (bits 16 down to 3) and a final constant 0 bit, which provides a one-cycle buffer for the slave to prepare for read accesses. Following the header, 64 cycles are used to read or write the data word, always transmitting from the Most Significant Bit (MSB) to the Least Significant Bit (LSB). Normally, single-word access is used for load and store instructions. Burst word access is used during the initialisation phase to initialise the program from external memory to the internal memory, which was discussed in the Init controller. Figure 3.11 illustrates the block diagram of the SPI master interface.

The module is designed as a Finite State Machine (FSM), and it is illustrated in Figure 3.8 with six distinct operational states to coordinate data shifting and manage strict timing:

S_IDLE The standby state. The module drives the Slave Select (`O_SS`) pin high and waits for a start trigger from the Memory Controller.

S_CS_LOW The initiation phase. The FSM pulls `O_SS` low for the first cycle of the transfer to select the external slave device. Simultaneously, the first bit of the header (the read/write indicator) is transmitted on the `O_MOSI` line.

S_HDR The address transmission phase. The FSM shifts out the remaining 15 bits of the header over the next 15 clock cycles.

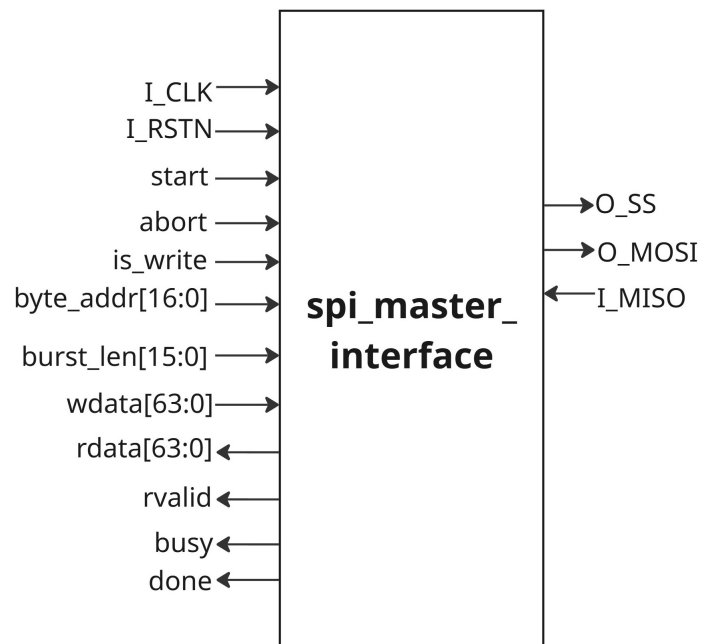


Figure 3.7: Block diagram of the SPI module.

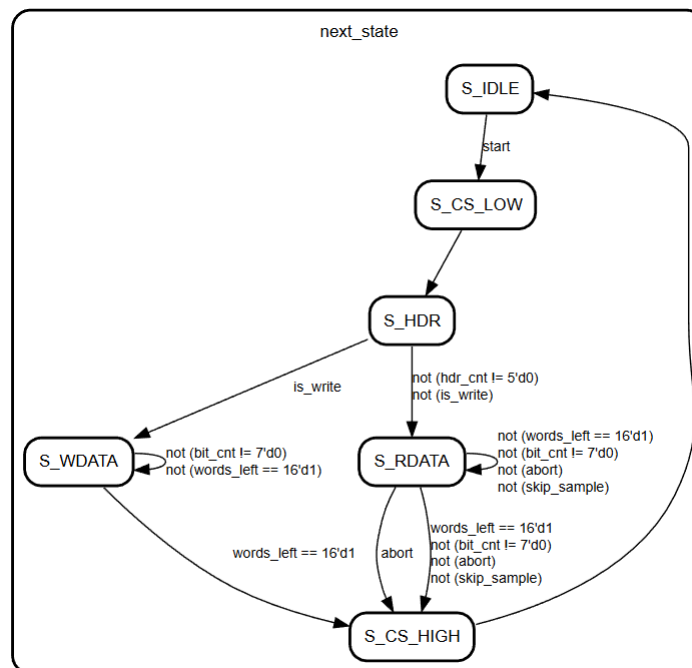


Figure 3.8: FSM of the SPI module.

S_WDATA The write phase. The 64-bit data word is shifted out sequentially on `O_MOSI`. If a burst write is configured, the FSM loops in this state, implicitly incrementing the byte address by 8 for each subsequent word until the `words_left` counter reaches zero.

S_RDATA The read phase. It begins with a dedicated dummy cycle (`skip_sample`) to satisfy the read transition requirement. Following this, it samples the `I_MISO` line for 64 cycles to construct the incoming data word. Like the write state, it natively supports burst reads by looping until the requested burst length is fulfilled.

S_CS_HIGH The termination phase. Once the final data word of a single or burst transfer is successfully transmitted or received, `O_SS` is driven high to conclude the transaction. The FSM asserts the `done` flag and returns to `S_IDLE`.

4 Functional Verification

4.1 Introduction to our verification framework

In order to thoroughly test our core, a verification framework is developed to generate random instruction streams and to verify the core with standard references. By using this framework, we are able to automatically detect all possible bugs when executing instructions and pinpoint the position in time where an error was first spotted.

4.2 Software components of the verification framework

Our framework integrates several industrial standard tools and self-developed programs to collaboratively verify the core. Figure 4.1 illustrates how the verification framework works. In the following sections, we will further describe how we utilized these tools for verification.

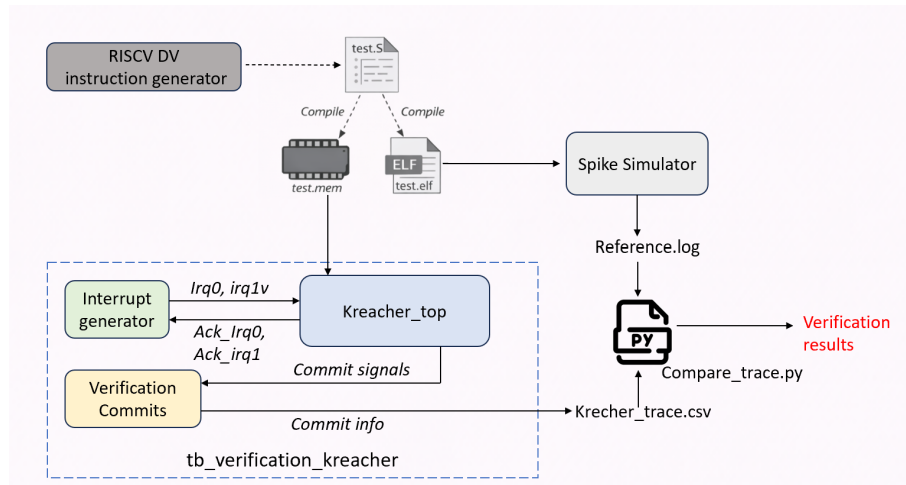


Figure 4.1: Illustration of the working principle of our verification framework

4.2.1 Random instruction generation tools

RISC-V DV (RISC-V Design Verification), originally developed by Google, is an open-source instruction generator used to test RISC-V processors.

RISC-V DV allows users to generate instructions tailored to their preferences by editing the riscv-dv test configuration. One can control the number of instructions, supported instruction types, supported ISA and such in YML files. Due to its flexibility, we were able to frequently use this tool to generate suitable assembly programs for tests of different extensions when developing our processor. The purpose is to generate assembly instructions that are randomized and long enough to detect as many hidden bugs as possible.

However, several instruction types are not generated by this framework in order to avoid potential conflicts. Because of this limitation, instructions from the Zicsr extension cannot be directly generated by the tool. For example, according to our specification, some CSR registers such as MISA and MTVEC are read-only. Therefore, when the target CSR is one of these registers, only read operations are allowed. To generate random Zicsr instructions, we developed an additional program that inserts legal CSR instructions into the original instruction stream. For instance, in the MSTATUS register, only the MIE and MPIE bits are writable. Consequently, only specific values (e.g., 0x80) are permitted when writing to this register using csrrs or csrrw instructions.

To ensure that only legal values are written, a valid value is first preloaded into a register that will later be used to update the mstatus register. An example is shown below:

```
1  li      x16, 0x80
2  csrrs x4, mstatus, x16
```

Furthermore, lw (load word) and sw (store word) instructions are inserted separately into the instruction stream. This is necessary for two main reasons. First, we must ensure that these instructions access addresses that can be mapped to valid memory regions in the SoC. Second, memory accesses must be properly aligned to prevent incorrect data reads or exceptions.

To guarantee this, a valid address is preloaded into a register before executing a load or store instruction. This value is then used by the subsequent load/store instruction to ensure that the target address is legal. An example is shown below. In this case, the marker "region_2" refers to an address entry located in external memory. We also pay attention that we only use the allowed special registers when we write compressed load/store instructions.

```
1  la x2, region_2 #region_2 points to an address specified in a linker
2  sw x12, -632(x2)
```

Within our instruction stream, we manually inserted a dedicated section to test whether the processor core runs correctly when store instructions modify instructions in program memory. This test case effectively verifies that the processor handles self-modifying codes properly when instructions are overwritten during program execution.

We present below a part of the test program that we developed. In the test program, several registers are first initialized with predefined values using li instructions. These values represent encoded machine instructions that will later overwrite existing instructions in the program memory. After initialization, the base address of the target instruction region, labeled STORE_WRITE, is loaded into registers x2 and x8 using the la instruction. A sequence of store operations is then executed to overwrite instructions starting at the STORE_WRITE label. Different store types, such as c.sd, sw, sb, are used to ensure coverage of all store word instruction types.

```
1  li x31, 0xa5210113a5210113
```

```

2          nop
3          li x13, 0x4151573b4151573b
4          nop
5          li x28, 0x41888fbb0300e03b
6          li x29, 0x5d9f78135d9f7813
7          la x2, STORE_WRITE
8          la x8, STORE_WRITE
9          sd x31, 0(x2)
10         c.sd x13, 8(x8)
11         li x12, 0x0293eb37da573317
12         c.sw x12, 12(x8)
13         c.sdsp x29, 16(x2)
14         c.swsp x28, 20(x2)
15 STORE_WRITE: c.addi      a7, -1
16             sraw        sp, t3, t1
17             sra         s3, s11, gp
18             c.addi      a7, -1
19             .....

```

In the assembly program above, the machine code of the four instructions located after "STORE_WRITE" are overwritten by 0xa5210113a5210113 after executing "sd x31, 0(x2)". 0xa5210113 translates to instruction "addi x2, x2, -1454". So when the CPU hits the address at "STORE_WRITE", we verify that the CPU executes "addi x2, x2, -1454" twice. With this test case, we are able to demonstrate that the Soc is capable of resolving conflicts when the same macro is being accessed for instruction read and store write at the same time, and that the core correctly supports instruction memory modifications.

At the final stage, We generated around 3000 instructions in order to reach sufficiently high coverage for possible cases. We are about to reach high coverage with our testcase, and we will present the results later.

4.2.2 Riscv-unknown-elf-gcc cross-compiler

Riscv-unknown-elf-gcc is a cross-compiler used to compile programs for RISC-V IAS based processors. This tool is essential for us to generate memory files for simulation.

We use the following command to compile the assembly file test_program.s into a bare-metal ELF executable for the RISC-V RV64IMC architecture with the Zicsr. The ELF file is compiled from an assembly program with random instructions, so there is no standard libraries or startup files needed for compilation. A custom linker script (linker.ld) is used, setting the starting address of the test program to 0x0000. Size optimization is enabled so that C instructions can be generated.

```

1 riscv64-unknown-elf-gcc -march=rv64imc_zicsr -mabi=lp64 -nostdlib
2 -nostartfiles -ffreestanding -fno-pie -no-pie -Os -mrelax -Wl,--relax
3 -Wl, -T,linker.ld test_program.s -o test_program.elf

```

Later, we use riscv64-unknown-elf-objcopy to convert the before generated ELF executable test_program.elf into a raw binary file test_program.bin for a RISC-V system.

```
1 riscv64-unknown-elf-objcopy -O binary test_program.elf test_program.bin
```

Then we use the following command to convert the binary file to a memory file. Xxd outputs the binary as 4-byte hex words, and sed reverses the byte order to match little-endian format. The memory file contains machine codes of the instructions in the assembly program. It is used to instantiate the external memory for simulation.

```
1 xxd -p -c 4 test_program.bin |
2 sed 's/\(..\)\\(..\)\\(..\)\\(..\)/\4\3\2\1/' > DMEM_content.mem
```

In our assembly test program, we used pseudo-instructions that do not exist in real RISC-V ISA. They are automatically translated into one or more real instructions by the compiler. To observe the real instructions that are being executed by the processor, we use the following command to convert the binary file to a readable file.

```
1 riscv64-unknown-elf-objdump -D -b binary -m riscv
2 test_program.bin > instructions.txt
```

4.2.3 Verification_commits module

We developed a module called "verification_commits" for verification. It is used to observe and record committed instructions from our 5-stage pipelined processor. It can also be simply modified to apply on the single cycle core. We connect it as a probe to the system to extract relevant execution information from the processor. We then feed the information to our verification framework, where the results can be automatically compared with our reference model Spike (Spike will be introduced later). In essence, the module captures the program counter (PC), instruction, register writes, and CSR writes when an instruction commits in the Write Back stage.

This module extracts information with the following ports:

Input Signals (From Pipeline)

```
IF_PC [XLEN-1:0]
IF_canonical_instruction [31:0]
WB_regfile_we
WB_csr_we
WB_RD_addr [4:0]
WB_RD [XLEN-1:0]
WB_csr_addr [11:0]
WB_csr [XLEN-1:0]
```

Output Signals (To Framework)

```
commit_PC [XLEN-1:0]
commit_instruction [31:0]
commit_valid
commit_csr_valid
commit_rd_addr [4:0]
commit_rd_value [XLEN-1:0]
commit_csr_addr [11:0]
commit_csr_value [XLEN-1:0]
```

Input ports "IF_PC" and "IF_canonical_instruction" are fetched from the Instruction Fetch stage. The rest of the input ports are extracted from the Write Back stage. For verification of our pipelined core, the information must be collected in the Write Back stage because this is where an instruction finally commits its effects. Since the PC and the instruction are only available in the Instruction Fetch stage,

and commits happen in the Write Back stage, we must propagate these signals through 3 registers so that they are aligned with the instruction at WB stage. Since the rest of the signals are collected at the final stage, no extra register is needed. Additionally, we do not record information of jump or branch instructions which does not reach Write Back stage. However, by verifying the order of PC, we can still learn that these instructions are executed correctly.

We collect the following information in realtime as the processor runs: current PC, the instruction being executed, write enable signal of the normal register bank and CSR bank, the address of the normal register or CSR being written, and the corresponding value that is written. These information is sufficient to describe the behavior of the core, and if the recorded behavior of our core matches the reference model, we can trust that our core is right.

4.2.4 Interrupt Generator

To test whether our core is properly handling interrupts, an interrupt generator is designed and connected with the system. The interrupt generator is started after the Soc finishes initialization. In the module, we have two interrupt signals, and each interrupt signal has a countdown counter. The counter values come from predefined sequences. These sequences act as deterministic pseudo-random delays between interrupts. When the counter reaches 0, the module asserts the interrupt signal and enters a WAIT_ACK state. Until ack is detected the interrupt signal remains in the WAIT_ACK state and the line stays high. The module is implemented to conform with task requirements.

4.2.5 Top Level Testbench

The module `tb_top_wrapper` is a simulation testbench we designed to verify the Soc. In the testbench, we instantiate the system (`top_wrapper`) as the device under test (DUT). Interrupt generator is connected to the system to simulate external interrupts. The `verification_commits` unit is also connected to monitor commit information from the processor. Whenever an instruction commits, the trace of its execution is written to a CSV file called `"kreach_trace.csv"`. Figure 4.2 shows part of the content of the csv file. `"kreach_trace.csv"` records only traces from normal program and does not record that of interrupt handlers.

Figure 4.3 presents a waveform from a simulation of the testbench in Vivado. It shows the information we collect in the csv file. The waveform displays data in a visible way, which is also helpful in aiding our verification.

When the core enters an interrupt handler, to conveniently observe the handling of interrupts and illegal instruction, another csv file called `"irq_trace.csv"` is also produced in our testbench. Figure 4.4 shows the content of this file. In this file we can easily read interrupt signals, acknowledge signals, mie bit and mcause value to check whether interrupt and trap behaviors are right.

Finally, at the end of the simulation, the testbench writes the contents of the external memory (DMEM) and 8 internal macros to `"DMEM_result.mem"` and `"PMEM_result_macro_index.mem"`. These memory files are used later to test whether store word instructions are writing the correct data.

time_ns	pc	inst	rd	rd_value	csr	csr_value
7865325	0x00002034	0x00000013	0	0x0		
7865355	0x00002036	0x00000013	0	0x0		
7865385	0x00002038	0x080a6a13	20	0x80		
7865415	0x0000203c	0x300a28f3	17	0x1800	768	0x1880
7865445	0x00002040	0x78c07037	0	0x78c07000		
7865475	0x00002044	0x3600001b	0	0x360		
7865505	0x00002048	0x00c00093	1	0xc		
7865535	0x0000204a	0x00100137	2	0x100000		
7865565	0x0000204e	0xbdd1011b	2	0xffbdd		
7865595	0x00002052	0x00c11113	2	0xffbdd000		
7871295	0x00002054	0x53510113	2	0xffbdd535		
7871355	0x0000205c	0x0003e1b7	3	0x3e000		
7871385	0x00002060	0xa371819b	3	0x3da37		
7871415	0x00002064	0x00e19193	3	0xf68dc000		
7871445	0x00002066	0x04618193	3	0xf68dc046		
7871475	0x0000206a	0x00a00213	4	0xa		
7871505	0x0000206c	0x300877f3	15	0x1880	768	0x1880
7871535	0x00002070	0x0010029b	5	0x1		
7871565	0x00002074	0x01f29293	5	0x80000000		
7871595	0x00002076	0x00015337	6	0x15000		
7871625	0x00002078	0xf653031b	6	0x14f65		
7871655	0x0000207c	0x00f31313	6	0xa7b28000		
7871685	0x0000207e	0xdcc30313	6	0xa7b27dcc		
7871715	0x00002082	0x00000393	7	0x0		

Figure 4.2: The trace record of executed instructions in the csv file

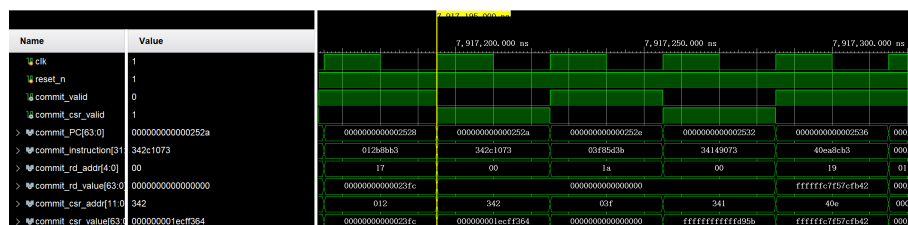


Figure 4.3: A waveform from a simulation showing important information of the core

time_ns	pc	inst	irq_0	irq_1	ack_0	ack_1	commit_valid	csr_valid	mie	mcause
8287785	0x000035c0	0x41d00133	1	1	0	0	0	0	1	3
8287815	0x000035c0	0x41d00133	1	1	0	0	1	0	1	3
8287845	0x000035c4	0x34239073	1	1	0	0	0	1	1	3
8287875	0x000035c8	0x02e902b3	1	1	1	0	1	0	0	0
8287905	0x00000000	0x00000000	1	1	0	0	0	0	0	0
8287935	0x00000000	0x00000000	1	1	0	0	0	0	0	0
8287965	0x00000000	0x00000000	0	1	0	0	0	0	0	0
8287995	0x00000000	0x0000206f	0	1	0	0	1	0	0	0
8288025	0x00000000	0x00000000	0	1	0	0	0	0	0	0
8288055	0x00000000	0x00000000	0	1	0	0	0	0	0	0
8288085	0x00002000	0x0100006f	0	1	0	0	1	0	0	0
8288115	0x00000000	0x00000000	0	1	0	0	0	0	0	0
8288145	0x00000000	0x00000000	0	1	0	0	0	0	0	0
8288175	0x00002010	0x2d738393	0	1	0	0	1	0	0	0
8288205	0x00002014	0xd2938393	0	1	0	0	1	0	1	0
8288235	0x00002018	0x30200073	0	1	0	0	0	0	1	0
8288265	0x00000000	0x00000000	0	1	0	0	0	0	1	0
8288295	0x00000000	0x00000000	0	1	0	0	0	0	1	0
8288325	0x000035cc	0x00027117	0	1	0	1	1	0	0	1

Figure 4.4: The interrupt trace record of the core in the csv file

4.2.6 Spike simulator

The Spike (RISC-V ISA Simulator) is the official instruction simulator for RISC-V processors. It executes RISC-V instructions and models the behavior of a RISC-V CPU. It is maintained by the RISC-V International community and designed to strictly follow the official RISC-V ISA specification, so it reflects the intended function. Due to its trustworthiness, it is widely used in industry and academia as the golden reference for verification. In our verification framework, we also rely on Spike to produce the reference trace for comparison.

We use the following command to configure and run Spike in our terminal. Spike uses the ELF file we generated before with Riscv-unknown-elf-gcc cross-compiler to simulate the assembly program. The option "log-commits" asks Spike to print a log of each committed instruction. "Instruction" sets the maximum number of instructions to be executed by Spike. "RV64imc_zicsr" is the supported ISA extension for running simulation. Spike uses 0x80000000 as the default starting program counter as 0x80000000 is the standard base address of RAM in industry. "PC" option allows the user to reset the starting program counter. We set the starting PC to be 0x2008 because Spike reserves the memory region 0x0000–0x2000 by default. This is because Spike places a small boot ROM and initialization code in this region for setting up the initial machine state before jumping to the program entry point. Due to this limitation, we changed the linker script and place the start of .text section to 0x2000 and we temporarily changed the reset PC to 0x2008 in hardware to align with Spike. We use this PC only for functional verification when doing co-simulation with Spike, and starting PC is set back to 0x0008 after successful verification according to task requirements. We know that the change of starting PC does not affect mapping of address and functionality of the core. Therefore, if the core passes verification with starting PC at 0x2008, we can safely say that it will also work on 0x0008. Option "m0" defines the memory region used for simulation. Option "priv" directs Spike to run the program in machine mode only. Under machine mode, MPP of mstatus register is set to "11", and the rest of the bits are hardwired to 0. Only TW, MIE and MPIE bits are writable. In our hardware implementation, we have similar settings, except that TW bit is hardwired to 0. So in our assembly test program we avoid only instructions that write 0 to TW to align with Spike. Under this mode, misa register stores the same value as our design. Registers mepc and mcause are also writable.

```
1 spike --log-commits --instructions=10000 --isa=rv64imc_zicsr --pc=0x2008 -
    m0x00002000:0x30000 --priv=m test_program.elf 2>&1 | tee spike.log
```

The following block shows a snapshot of the log printed by Spike simulation. As we can see, the fourth column is the current PC. The fifth column is the corresponding instruction code. There are 4 forms of traces. "mem 0x0000000000005d30 0x00000000" means that 0x00000000 is being written to address 0x0000000000005d30, indicating a store word instruction. "x10 0x00000000000024ac" means Spike writes 0x24ac to x10. Trace like "x13 0x000000002b121fc8 mem 0x00000000002a010" indicates a load word instruction. It means that data 0x000000002b121fc8 is read from address 0x2a010 and is loaded into x13. "x7 0x0000000000001880 c768_mstatus 0x0000000000001880" is related to zicsr instructions. At this PC, 0x1880 is written into mstatus register and the original value 0x1880 inside c768_mstatus is written into x7.

```

1 core 0: 3 0x00000000000024a6 (0xdc38) mem 0x0000000000005d30 0x00000000
2 core 0: 3 0x00000000000024a8 (0xfd5ff56f) x10 0x00000000000024ac
3 core 0: 3 0x000000000000247c (0x048000ef) x1 0x0000000000002480
4 core 0: 3 0x00000000000024c4 (0xfb1ffb6f) x22 0x00000000000024c8
5 core 0: 3 0x0000000000002474 (0x01e000ef) x1 0x0000000000002478
6 core 0: 3 0x0000000000002492 (0x00028417) x8 0x0000000000002a492
7 core 0: 3 0x0000000000002496 (0xb6e40413) x8 0x0000000000002a000
8 core 0: 3 0x000000000000249a (0x4814) x13 0x000000002b121fc8 mem 0
   x0000000000002a010
9 core 0: 3 0x000000000000249c (0x030000ef) x1 0x00000000000024a0
10 core 0: 3 0x00000000000024cc (0xffdff0ef) x1 0x00000000000024d0
11 core 0: 3 0x00000000000024c8 (0x024000ef) x1 0x00000000000024cc
12 core 0: 3 0x00000000000024ec (0x00a4d993) x19 0x003fffffffffffffff
13 core 0: 3 0x00000000000024f0 (0x305062f3) x5 0x0000000000000001
14 core 0: 3 0x00000000000024f4 (0x4b81) x23 0x0000000000000000
15 core 0: 3 0x00000000000024f6 (0x4e81) x29 0x0000000000000000
16 core 0: 3 0x00000000000024f8 (0x08000093) x1 0x0000000000000080
17 core 0: 3 0x00000000000024fc (0x300093f3) x7 0x0000000000001880
   c768_mstatus 0x0000000000001880

```

With the trace log from Spike as reference, as well as trace log we obtain from simulation using our testbench, we are able to verify our core from a microscopic view in an automated way. While spike is powerful, it has its limitation. Spike can't be used for interrupt handling and illegal instruction handling. So when running on Spike, we replaced wfi instructions and illegal instructions with nop instruction as placeholder, and when we compare the trace from Spike with the one from hardware simulation, we ignore traces from handlers. We make sure that register x1-x31 is not changed when entering and leaving a handler. From verification of correct interrupt handling, we referred to irq_trace.csv generated with our testbench and performed manual check.

4.2.7 Trace comparison python script

We developed a compare_trace.py for automatically comparing executive trace information from spike and RTL simulation and generating verification results.

This python file does the following things: First, it reads the log generated by Spike and extracts relevant information; Second, it reads the CSV trace produced by the hardware simulation; then, it checks whether both traces match one instruction at a time; Third, it checks whether data memory (DMEM) and program memory (PMEM) contents after simulation match the expected results.

When executing this python script, if a difference is found between two traces, the script prints the first mismatch location and the reference value versus simulated value. Figure 4.5 below shows the content printed by the software when a mismatch is found. By doing so, we can easily identify where the core first goes wrong and which instruction might not be properly executed. This information allows us to easier pinpoint the bugs and to fix issues more efficiently.

When the verification passes, we should be able to see the content in figure 4.6. The smiley face is the reward for a successful verification.

```

First mismatch at item 1373
Simulated PC:0x3982
Reference PC:0x3982
Simulated instruction:0xbb418db7
Reference instruction:0xbb418db7
Simulated wb_reg:27
Reference wb_reg:27
Simulated wb_val:0xffffffff
Reference wb_val:0xffffffffbb418000
Assembly instruction: lui s11,0xbb418
Traceback (most recent call last):
  File "/mnt/d/riscv/vlsi_processor_design_project/0_Verification/auto_test/compare_trace.py", line 621, in <module>
    raise ValueError("A mismatch has been found.")
ValueError: A mismatch has been found.

```

Figure 4.5: Illustration of the content printed by the python file when a mismatch is found

```
PC matched.  
Instructions matched.  
Write back registers matched.  
Write back values matched.  
Store word instructions checked. DMEM result matched.  
Store word instructions checked. PMEM_0 result matched.  
Store word instructions checked. PMEM_1 result matched.  
Store word instructions checked. PMEM_2 result matched.  
Store word instructions checked. PMEM_3 result matched.  
Store word instructions checked. PMEM_4 result matched.  
Store word instructions checked. PMEM_5 result matched.  
Store word instructions checked. PMEM_6 result matched.  
Store word instructions checked. PMEM_7 result matched.  
Store word instructions checked. All PMEM results matched.  
Congratulations! You have survived the torture test!
```


##

}}}

== ==
== ==
== ==
==

Figure 4.6: Illustration of the printed content when the verification is passed

4.3 Coverage result

In image 4.8, we can see the coverage result of our single cycle core(without toggle). And in figure 4.9 some detailed coverage information is displayed. Figure 4.7 and ?? show the overall and detailed coverage result of the pipelined core respectively. It can be concluded from the results that We achieved relatively high coverage(93% in total for single cycle core and 95% for pipelined core) with our test program. Our IF (Instruction Fetch) stage, ID (Instruction Decode) stage, EX (Execution) stage and MEM (Memory) stage and HCU (Hazzard Control Unit) all achieved more than 90% coverage. Only several corner cases that are unlikely to occur are not tested. The high coverage is due to a large number of instructions (around 3000), the use of random instruction generator and the extra instructions that we added.

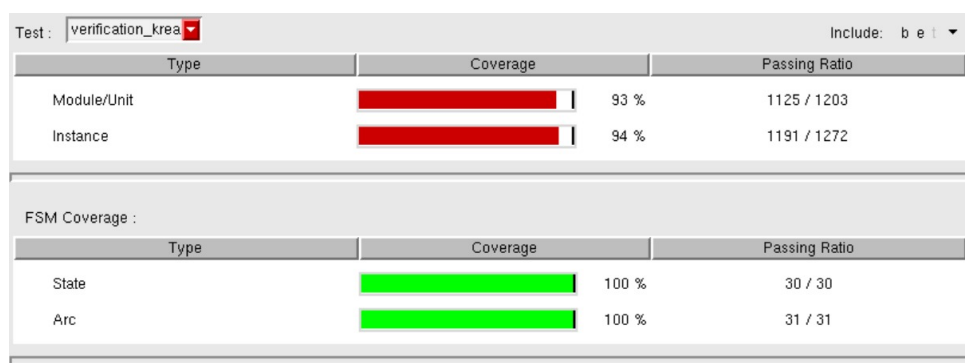


Figure 4.7: Coverage result of the pipelined core using our assembly test program

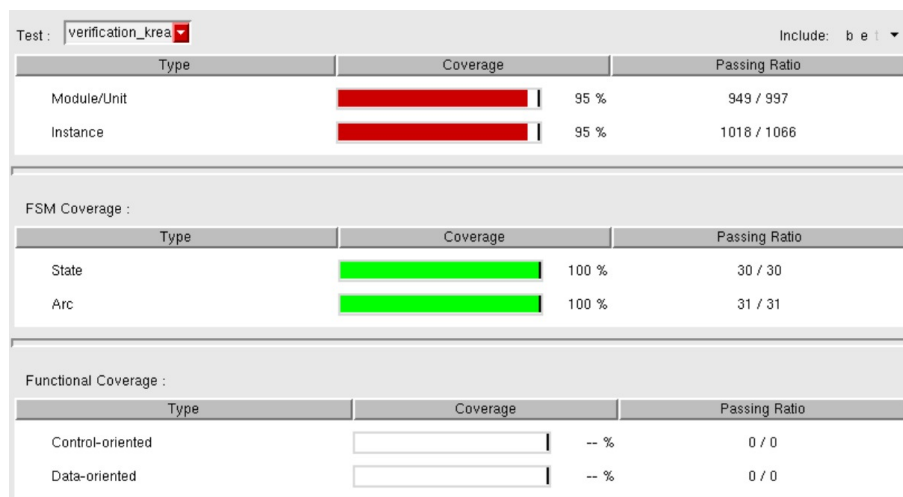


Figure 4.8: Coverage result of the single cycle core using our assembly test program

Instance	Types	Self Total	Cumulative Total
verification_kreacher	bet	--- (0 / 0)	95% (1018 / 1066)
external_memory	bet	87% (7 / 8)	87% (7 / 8)
u_spi_slave	bet	88% (38 / 43)	88% (38 / 43)
u_kreacher_top	bet	100% (3 / 3)	96% (973 / 1015)
pads_i	bet	--- (0 / 0)	--- (0 / 0)
spi_master_interface_inst	bet	93% (27 / 29)	93% (27 / 29)
core_inst	bet	--- (0 / 0)	95% (688 / 720)
u_MEM	bet	--- (0 / 0)	81% (52 / 64)
u_IF_HK	bet	--- (0 / 0)	95% (146 / 153)
u_ID	bet	100% (7 / 7)	95% (196 / 206)
u_interrupt_handler	bet	98% (46 / 47)	98% (46 / 47)
u_EX	bet	92% (12 / 13)	99% (230 / 232)
u_pause_handler	bet	100% (11 / 11)	100% (11 / 11)
u_WB	bet	100% (7 / 7)	100% (7 / 7)
init_memory_controller_inst	bet	--- (0 / 0)	97% (103 / 106)
PMEM_interface_top_inst	bet	--- (0 / 0)	97% (152 / 157)

Figure 4.9: Detailed coverage information of submodules of the single cycle core

Instance	Types	Self Total	Cumulative Total
verification_kreacher_pipelined	bet	--- (0 / 0)	94% (1191 / 1272)
external_memory	bet	87% (7 / 8)	87% (7 / 8)
u_spi_slave	bet	88% (38 / 43)	88% (38 / 43)
u_kreacher_top	bet	100% (3 / 3)	94% (1146 / 1221)
pads_i	bet	--- (0 / 0)	--- (0 / 0)
spi_master_interface_inst	bet	93% (27 / 29)	93% (27 / 29)
core_inst	bet	--- (0 / 0)	94% (867 / 926)
u_MEM	bet	100% (7 / 7)	66% (47 / 71)
u_WB	bet	86% (6 / 7)	86% (6 / 7)
u_interrupt_handler	bet	92% (62 / 67)	92% (62 / 67)
u_ID	bet	94% (31 / 33)	95% (220 / 232)
u_IF_HK	bet	100% (13 / 13)	96% (182 / 189)
u_EX	bet	90% (36 / 40)	97% (258 / 266)
u_HCU	bet	97% (81 / 83)	97% (81 / 83)
u_pause_handler	bet	100% (11 / 11)	100% (11 / 11)
init_memory_controller_inst	bet	--- (0 / 0)	94% (100 / 106)
PMEM_interface_top_inst	bet	--- (0 / 0)	95% (149 / 157)

Figure 4.10: Detailed coverage information of submodules of the pipelined core

5 Synthesis and Physical Design

5.1 Synthesizability check of Kreacher SoC

We analyzed the Kreacher SoC RTL using the Cadence HAL RTL checker. All initial errors and critical warnings were successfully resolved, yielding a clean verification report.

While the tool did flag several minor warnings such as LCVARN, DALIAS, FDTHRU, INTTOB, and URDWIR—these largely represent intentional design choices rather than actual flaws. For example, LCVARN warnings simply point to naming aliases that do not affect functionality. Similarly, the FDTHRU warnings correctly identify feedthrough paths from input to output, which we intentionally implemented. Because linting tools often flag unconventional but functionally correct configurations, it is essential to evaluate these messages within the context of the design intent. After a thorough review, we confirmed that none of these remaining warnings compromises the reliability of the SoC.

5.2 Synthesis Results

To enable the chip to communicate with the outside world, pad cells are logically instantiated prior to synthesis. For this design, a bidirectional pad (HIO18_GF28SLP_IOPAD) was utilized, with each pad cell tied to an OE18 signal (HIO18_GF28SLP_OETIE).

During the logic synthesis phase, Synopsys Design Compiler transforms the high-level circuit design into a fully technology-mapped netlist. The tool executes a systematic workflow: it parses the HDL files, performs logic minimization and optimization, and concludes with technology decomposition and mapping. To guide this process, the target library was set to hpsnlib_g28_9t_RVT, supported by the hpsnio_g28_HIO18 IO pad library and the hpsnmem_g28_HM_1P_GF28SLP SRAM macro library.

Following the library setup, the worst-case, typical, and best-case operating conditions were defined for synthesis. Finally, critical design constraints such as the target clock period, input/output delays, and clock latency—were applied via the constraints.tcl file, as illustrated in Figure 5.1.

We iteratively fine-tuned the input and output delays alongside the target clock frequency to optimize the synthesis results. Following synthesis, the tool generated comprehensive timing, area, and power reports. The timing report highlighted the critical paths across register-to-register (reg2reg), input-to-register (in2reg), and register-to-output (reg2out) logic. Crucially, the design had to demonstrate positive slack across these paths to confirm the absence of timing violations. Finally, the area and power reports were analyzed to establish pre-routing estimates for the design's physical footprint and power consumption. The final synthesis results for timing, area, and power are summarized in Table 5.1.

5.2.1 Timing report analysis

The timing report shows positive or zero slack on all three critical paths with the "hpsnlib_g28_9t_RVT" library. They are illustrated in Figures 5.2 with Reg2Reg slack 0, 5.3 with in2Reg slack 0, and 5.4 with

```

1 #####
2 # definition of clocks
3 #####
4 create_clock -period 11.8 -waveform { 0 5.9 } [get_ports {I_CLK}] -name CLK
5 set_clock_transition 0.3 [get_clocks {CLK}]
6 set_clock_uncertainty 0.2 -setup [get_clocks {CLK}]
7 set_clock_latency 0.5 [get_clocks {CLK}]
8
9 #####
10 # Set I/O delays
11 #####
12 set_output_delay -clock [get_clocks {CLK}] 5.9 [all_outputs]
13 set_input_delay -clock [get_clocks {CLK}] 5.9 [remove_from_collection [all_inputs] [get_ports "I_CLK"]]
14
15 #####
16 # Set input, inout and output port conditions
17 #####
18 # Additional load values for all inputs
19
20 set_load 1.000 [all_inputs]
21
22 # Capacitance for all output or inout ports.
23 set_load 1.000 [all_outputs]
24
25 #
26 # Design attributes
27 #
28 set_dont_touch_pads_i/i_HIO*
29 set_dont_touch_pads_i/*_pad*
30

```

Figure 5.1: Synthesis constraints specified in the constraints.tcl file.

Reg2out slack 0.

data arrival time	0.00	12.02 r
		12.02
clock CLK (rise edge)	11.80	11.80
clock network delay (ideal)	0.50	12.30
clock uncertainty	-0.20	12.10
core_inst/u_ID/u_csr_bank/mepc_reg_reg_1_/CP (H_9T_RVT_DFPRQX2)	0.00	12.10 r
library setup time	-0.07	12.03
data required time		12.03

data required time		12.03
data arrival time		-12.02

slack (MET)		0.00

Figure 5.2: Timing Report(REG2REG).

data arrival time		8.74
clock CLK (rise edge)	11.80	11.80
clock network delay (ideal)	0.50	12.30
clock uncertainty	-0.20	12.10
init_memory_controller_inst/u_init_ctrl/state_reg_0_/CP (H_9T_RVT_DFPRQX1)	0.00	12.10 r
library setup time	-0.17	11.93
data required time		11.93

data required time		11.93
data arrival time		-8.74

slack (MET)		3.20

Figure 5.3: Timing Report(in2REG).

5.2.2 Power report analysis

As observed in Table 5.1, power consumption scales proportionally with clock frequency. This is an expected trend, as higher frequencies dictate faster charge and discharge cycles, leading to increased dynamic switching activity within the circuit. For example, the total power consumption is 208.9862 μ W at a frequency of 84.74 MHz.

Clock Period(ns)	Slack(ns)	Data path(ns)	Power (mW)	Area(μm^2)
11.80 ns(84.74 MHz) clk(reg to reg) regin regout	0.00 3.20 0.00	12.02 ns	208.8034 uW	144228.358191 μm^2
10.5 ns(95.23 MHz) clk(reg to reg) regin regout	0.00 2.55 -0.78	10.73 ns	234.7433 uW	144572.104187 μm^2
10.22 ns(97.84 MHz) clk regin regout	-0.04 2.41 -0.87	10.44 ns	241.5398 uW	145231.867171 μm^2
10 ns(97.8 MHz) clk regin regout	-0.01 2.30 -1.01	10.22 ns	246.8224 uW	145338.570000 μm^2

Table 5.1: Summary of post-synthesis timing, area, and power metrics across various clock periods for pipelined SoC.

data arrival time		6.20
clock CLK (rise edge)	11.80	11.80
clock network delay (ideal)	0.50	12.30
clock uncertainty	-0.20	12.10
output external delay	-5.90	6.20
data required time		6.20

data required time		6.20
data arrival time		-6.20

slack (MET)		0.00

Figure 5.4: Timing Report(REG2out).

5.2.3 Area report analysis

Table 5.1 also illustrates a direct correlation between higher frequencies and increased chip area. As the frequency target is pushed higher, the synthesis tool must instantiate additional circuitry, such as larger drive cells and buffers, to satisfy the stricter timing constraints. To mitigate this area penalty, we implemented specific architectural optimizations within the RISC-V M-extension. Initially, relying on the default synthesis operators ($*$ and $/$) for multiplication and division yielded an excessively large footprint. We addressed this by replacing these standard operators with a dedicated radix-4 Booth multiplier and a radix-4 divider, which significantly optimized the overall area. The final area $144228.358191 \mu\text{m}^2$ with frequency 84.74 MHz is illustrated in the Figure 5.5.

Figure 5.6 illustrates the optimized schematic generated by the synthesis tool at an abstract, gate-level view. In contrast, the post-physical design schematic provides a much more precise representation of the circuit, fully accounting for physical routing, technology-node-specific design rules, and exact timing constraints.

```

22 Operating Conditions: wc_0d76V_m40C   Library: hpsnlib_g28_9t_RVT_wc_0d76V_m40C
23 Wire Load Model Mode: top
24
25
26 Global Operating Voltage = 0.76
27 Power-specific unit information :
28   Voltage Units = 1V
29   Capacitance Units = 1.000000ff
30   Time Units = 1ns
31   Dynamic Power Units = 1uW      (derived from V,C,T units)
32   Leakage Power Units = 1nW
33
34
35 Cell Internal Power      = 181.4186 uW   (87%)
36 Net Switching Power     = 27.3848 uW   (13%)
37 -----
38 Total Dynamic Power      = 208.8034 uW   (100%)
39
40 Cell Leakage Power      = 183.0015 nW
41
42
43
44 Power Group      Internal      Switching      Leakage      Total
45                  Power        Power          Power        Power
46 -----
46 io_pad           98.9460      9.9439        15.9836       108.9059 ( 52.11%)
47 memory           26.6594      0.5703        0.0000        27.2297 ( 13.03%)
48 black_box        0.0000      0.0000        0.0000         0.0000 (  0.00%)
49 clock_network    3.6138      2.1955        0.6620         5.8099 (  2.78%)
50 register         47.5378      0.1564        70.5438       47.7648 ( 22.86%)
51 sequential       0.0000      0.0000        0.0000         0.0000 (  0.00%)
52 combinational    4.6614      14.5190       95.7900       19.2758 (  9.22%)
53 -----
54 Total            181.4185 uW    27.3850 uW    182.9795 nW    208.9862 uW
55 1

```

Figure 5.5: Power Report.

```

9 Library(s) Used:
10   hpsnlib_g28_9t_RVT_wc_0d76V_m40C (File:
11     /proj/prz25/workspaces/group10/sapa892g/resources/hpsnlib_g28_9t_RVT/timing/hpsnlib_g28_9t_RVT_wc_0d76V_m40C.ccs.db)
12   HM_1P_GF28SLP_wc_0d76V_m40C (File:
13     /proj/prz25/workspaces/group10/sapa892g/resources/hpsnmem_g28_HM_1P_GF28SLP/timing/nldm/HM_1P_GF28SLP_wc_0d76V_m40C.db)
14   hpsnio_g28_HIO18_wc_0d76V_1d62V_m40C (File:
15     /proj/prz25/workspaces/group10/sapa892g/resources/hpsnio_g28_HIO18/timing/hpsnio_g28_HIO18_wc_0d76V_1d62V_m40C.db)
16
17 Number of ports:          9
18 Number of nets:          628
19 Number of cells:         10
20 Number of combinational cells: 1
21 Number of sequential cells: 4
22 Number of macros/black boxes: 0
23 Number of buf/inv:       0
24 Number of references:    7
25
26 Combinational area:      15511.974951
27 Buf/Inv area:           2154.789039
28 Noncombinational area:   11826.711365
29 Macro/Black Box area:    116889.671875
30 Net Interconnect area:   undefined (No wire load specified)
31
32 Total cell area:         144228.358191
33 Total area:              undefined

```

Figure 5.6: Area Report.

5.3 Physical Design

Post synthesis, the technology-mapped netlist moves to the back-end for physical design. This phase transforms the design into a physical layout through a sequence of core steps: floorplanning, placement, routing, and post-verification. We rely on Cadence Innovus to handle the detailed layout and optimization, ultimately generating the final GDSII file required by the foundry for fabrication.

Real-world physical constraints such as wire lengths, routing delays, and capacitive loading—heavily impact the maximum achievable clock speed. To account for this and avoid over-constraining the tool, we deliberately relaxed the SoC’s target operating frequency to 84.74 MHz. This adjustment allows for a much smoother and more reliable placement and routing process.

5.3.1 Bind

Properly defining constraints is a critical phase of the design process. The icfe bind command seamlessly links these parameters to the design environment, ensuring that the resulting synthesized netlist

accurately reflects and meets the specified timing requirements.

5.3.2 Floor Planning and Power Planning

Floorplanning and power planning are deeply interconnected stages in physical design. Floorplanning establishes the chip's physical dimensions and maps out the initial placement of major components, such as IPs and macros. These blocks are strategically arranged within the core area—the region dedicated to logic cells—while the surrounding die area defines the chip's total footprint. Concurrently, power planning establishes the power distribution network. This involves routing a power ring connected to the VDD and VSS pads, along with power straps designed to reduce parasitic resistance and inductance. Ultimately, this network ensures that all logic cells can draw reliable power locally, which is essential for controlling heat dissipation, minimizing IR drop, and preventing electromigration. In "parameters.tcl", the die area is specified as 589 x 589.

5.3.3 Placement

With the power distribution network in place, the next step is to populate the floorplan rows with standard cells. This process is divided into two main phases: Global Placement and Detailed Placement. Global Placement, sometimes called Fast Placement, prioritizes speed to find the general optimal neighborhood for each cell. Because this is a "soft" placement where cells aren't strictly locked down, it typically leaves behind minor overlaps or row misalignments. Detailed Placement then takes over to legalize the layout. It resolves any overlaps and snaps the cells into precise, valid locations while respecting the constraints set during the global phase. For this implementation, macro positions were explicitly defined beforehand, and existing scripts were used to drive the overall placement run.

5.3.4 Clock Tree Synthesis

A primary goal in the design process is to minimize clock latency, ideally driving it down to zero. This guarantees that every register receives the clock signal at the exact same time. If the timing is off, registers might capture data at the wrong moments, leading to functional errors. To prevent this, a clock distribution tree is built to route the clock signal to all registers while keeping clock skew as close to zero as possible. During this stage, clock buffers are inserted along the paths to maintain signal integrity and boost driving strength. Standard algorithms, such as H-trees and X-trees, are typically used to construct this network efficiently.

5.3.5 Routing

Now that the blocks are placed, the routing phase connects them using actual physical wires. Up until this point, timing analysis relied on an ideal scenario that assumed zero wire delay. With physical wires laid out, timing must be re-analyzed using real-world delays to ensure no new violations have emerged. To guarantee the chip can be successfully fabricated, Design Rule Checks (DRC) are strictly enforced throughout this process. This stage also introduces several other critical verifications, including signal integrity (SI) evaluation, cross-talk analysis, detailed power analysis, and diode insertion to prevent antenna effects on the metal layers. These rigorous checks are essential for maximizing the performance, reliability, and manufacturability of the final design.

5.3.6 Signoff

In this final step, filler cells are inserted into any remaining gaps between standard cells to ensure physical continuity across the rows. A comprehensive final sign-off is then conducted, verifying that there are no unrouted nets and that the design strictly adheres to all Design Rule Checks (DRC), signal integrity standards, and timing constraints. Once these rigorous checks are cleared, the finalized layout is exported as a GDSII file, marking its readiness for tape-out and subsequent fabrication. Figures 5.7 and 5.8 depict the final layout of the pipelined core SoC and single-cycle SoC after the physical design.

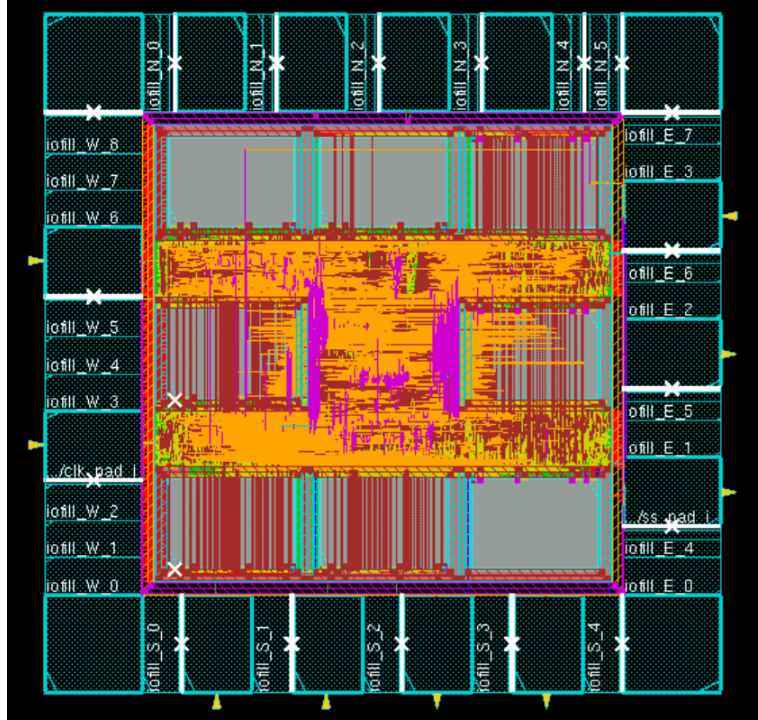


Figure 5.7: Chip layout with pipelined core.

5.4 Discussion and Conclusion

The primary objective of this project is to design, implement, and verify a high-performance RISC-V-based System-on-Chip (SoC). To achieve our core design goal of maximizing performance, the final architecture transitioned from a single-cycle implementation, which operated with a 30 ns clock period (33.3 MHz), to a 5-stage pipelined design. This architectural shift significantly enhanced the system's throughput, allowing the pipelined core to achieve a minimized clock period of 11.80 ns (84.74 MHz). This represents an approximate 2.5x increase in operational frequency over the single-cycle baseline. Through rigorous verification and post-layout validation, the pipelined Kreacher SoC proved to be functionally correct and highly robust, successfully fulfilling the project's performance and integration objectives.

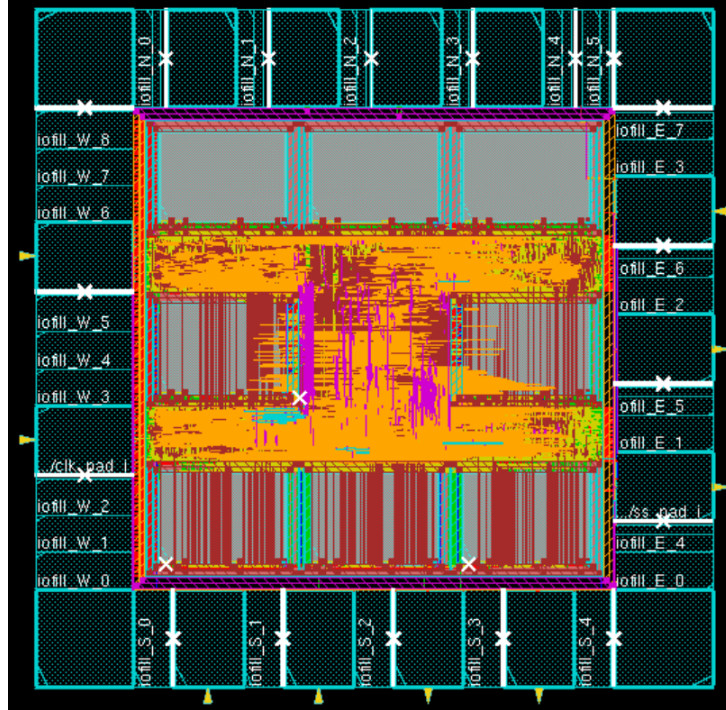


Figure 5.8: Chip layout with single-cycle core.

5.5 Future Work

While the current pipelined architecture successfully meets the design and performance targets, future iterations of the SoC can focus on further frequency scaling and optimization. The primary focus for this development will be identifying and shortening the critical paths within the pipelined core's logic. By specifically targeting and optimizing these timing bottlenecks in the critical path, the overall clock period can be reduced even further, unlocking higher clock speeds and greater computational performance for the system.

6 Validation

For validation of our SoC after place and route, we utilized four C programs for testing. All four programs can be seen in the appendix. The example program here performs 16×16 matrix multiplication and compresses the result into a signature. This is used to verify that our SoC executes arithmetic, memory, and control instructions correctly.

```

1  #include <stdint.h>
2  #if defined(__riscv)
3  #include "corelib.h"
4  __attribute__((section(".exit_value")))
5  volatile uint32_t exit_value;
6  #else
7  #include <stdio.h>
8  #endif
9  static uint32_t signature_test(void) {
10     static int32_t A[16][16];
11     static int32_t B[16][16];
12     static int32_t C[16][16];
13     for (uint32_t i = 0; i < 16; i++) {
14         for (uint32_t j = 0; j < 16; j++) {
15             A[i][j] = (int32_t)((i * 17u + j * 7u + 3u) & 0xffu) - 128;
16             B[i][j] = (int32_t)((i * 11u + j * 13u + 5u) & 0xffu) - 128;
17             C[i][j] = 0;
18         }
19     }
20     for (uint32_t i = 0; i < 16; i++) {
21         for (uint32_t j = 0; j < 16; j++) {
22             int64_t sum = 0;
23             for (uint32_t k = 0; k < 16; k++) {
24                 sum += (int64_t)A[i][k] * (int64_t)B[k][j];
25             }
26             C[i][j] = (int32_t)sum;
27         }
28     }
29     #if !defined(__riscv)
30     printf("Matrix multiplication result C=A*B:\n");
31     for (uint32_t i = 0; i < 16; i++) {
32         for (uint32_t j = 0; j < 16; j++) {
33             printf("%8d ", C[i][j]);
34         }
35         printf("\n");

```

```

36     }
37 #endif
38     uint64_t acc = 0x510e527fade682d1ULL;
39     for (uint32_t i = 0; i < 16; i++) {
40         for (uint32_t j = 0; j < 16; j++) {
41             acc ^= (uint64_t) (uint32_t) C[i][j];
42             acc += ((uint64_t) (uint32_t) C[i][j] << ((i + j) & 15u));
43
44             if (C[i][j] < 0) acc ^= 0x1111u;
45             else             acc += 0x2222u;
46
47             switch ((i + j) & 3u) {
48                 case 0: acc += 0x11111111ULL; break;
49                 case 1: acc ^= 0x22222222ULL; break;
50                 case 2: acc -= 0x33333333ULL; break;
51                 default: acc ^= (acc >> 11); break;
52             }
53         }
54     }
55     return (uint32_t) (acc ^ (acc >> 32));
56 }
57 int main(void) {
58     uint32_t sig = signature_test();
59     #if defined(__riscv)
60         exit_value = sig;
61     #else
62         printf("matmul_signature=0x%08x\n", sig);
63     #endif
64     return (int) sig;
65 }

```

We first run the program on the computer to get the correct signature. Later we use the make software provided in the work space to generate the memory picture of the program and load it into program memory for post-implementation simulation. Figure 6.1 shows the waveform of a memory cell. By checking that the final value at address 0x4000 in the macro, we validate the correctness of our SoC.

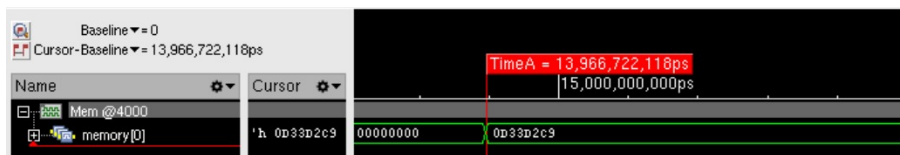
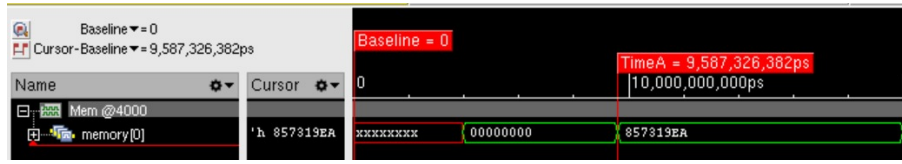
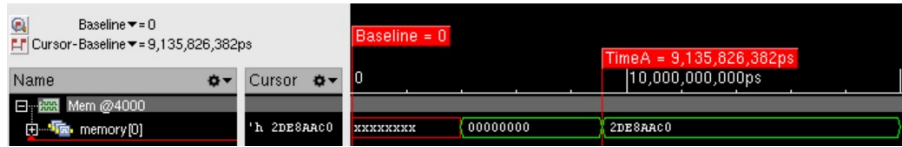


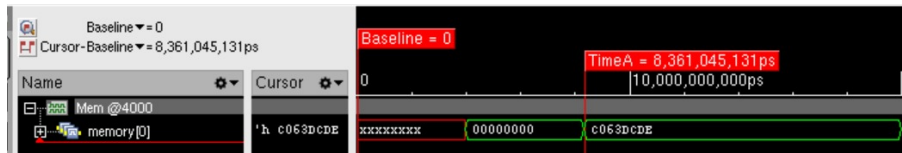
Figure 6.1: Waveform of the post-place-and-route system simulated with matmul_test.c

Figure 6.2 shows the simulation result of our SoC running on cpu_stress_test.c. The signature also matches the expected value.

We also have another program called "fft_test.c" which computes a 64-point Fast Fourier Transform (FFT) and generates a compressed signature. The result in figure 6.3 again validates our work.

Figure 6.2: Waveform of the post-place-and-route system simulated with `cpu_stress_test.c`Figure 6.3: Waveform of the post-place-and-route system simulated with `fft_test.c`

To be as comprehensive as possible in testing the system, we have one final program that implements AES-128 encryption. Our system again generates correct return value at 0x4000 as can be seen in figure 6.4. From the results of all 4 programs that we present here, we can confidently say that our post-place-and-route system is quite robust. Figure 6.5 shows that `synth_test.svwf` was used for testing.

Figure 6.4: Waveform of the post-place-and-route system simulated with `aes128_test.c`

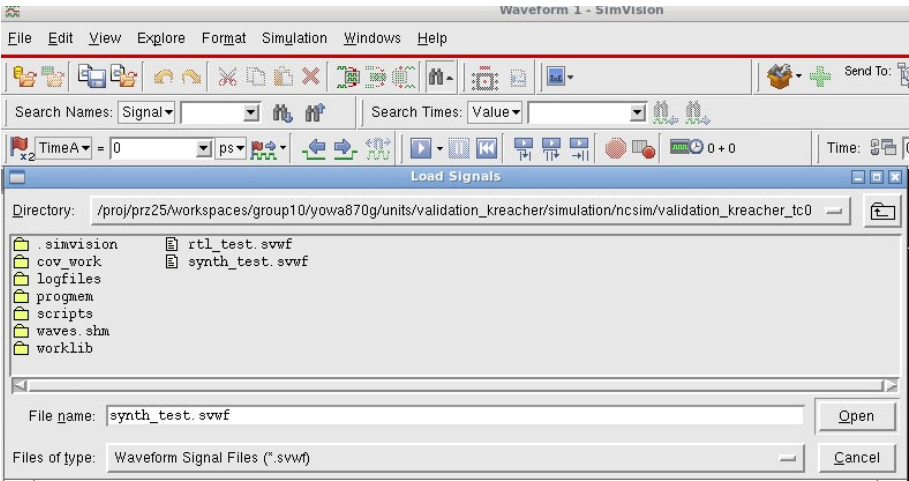


Figure 6.5: synth_test.svwf loaded in SimVision for simulation testing

7 Appendix

Figure 7.1 is the schematic of the entire single cycle core.

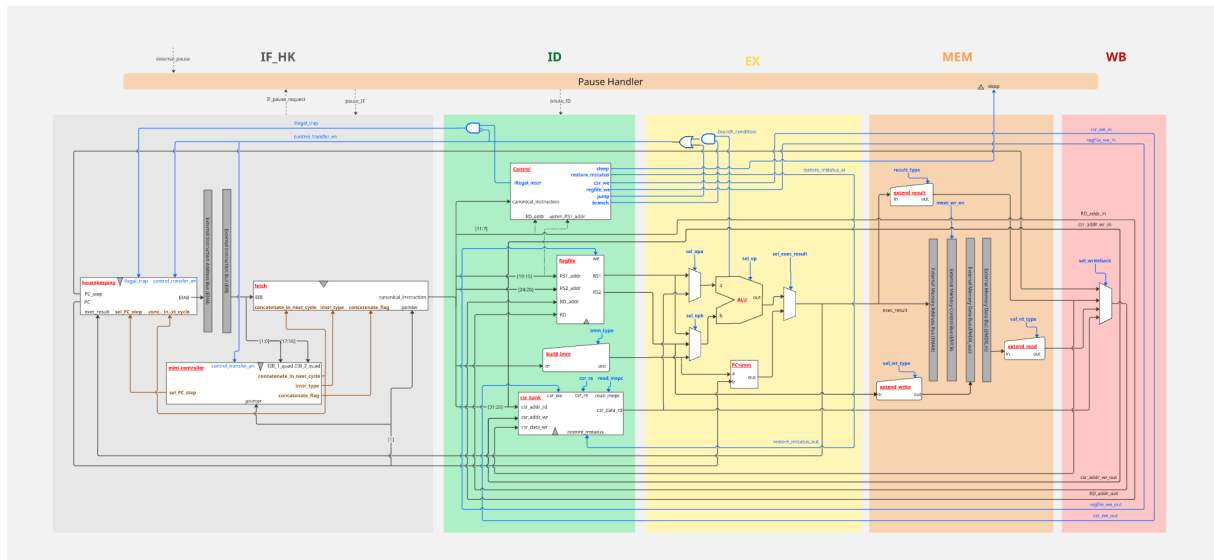


Figure 7.1: Complete schematic of the single cycle core

Figure 7.2 is the schematic of the entire pipelined core.

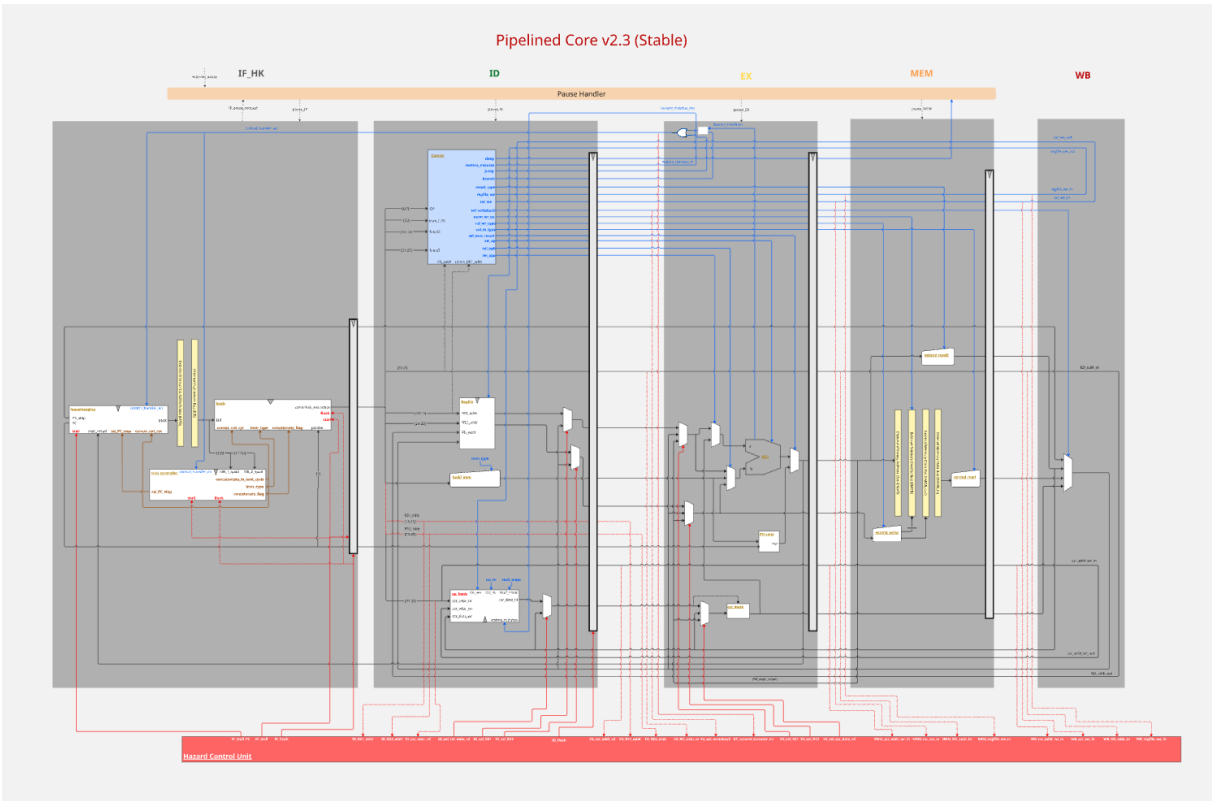


Figure 7.2: Complete schematic of the pipelined core

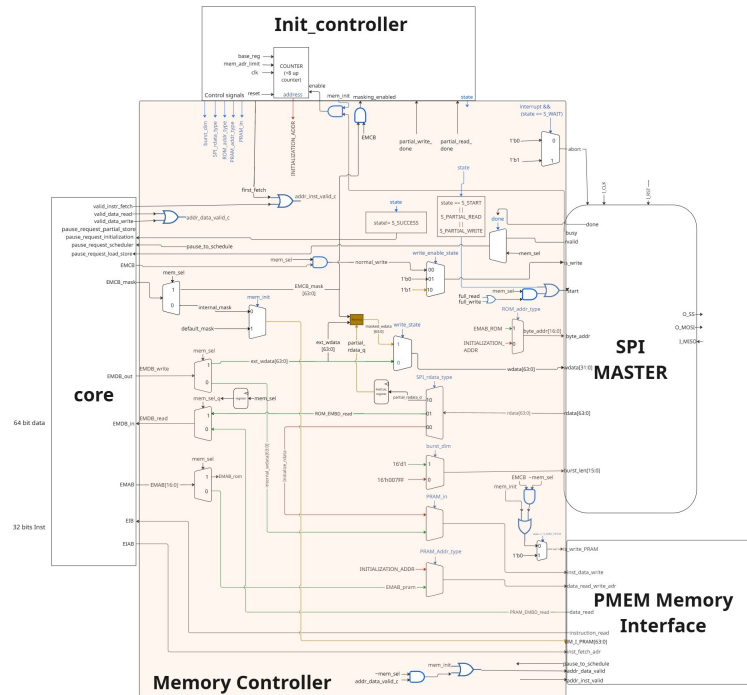


Figure 7.3: Schematic diagram of the Init Memory Controller.

The following program is `cpu_stress.test.c` for validation.

```

1  #include <stdint.h>
2
3  #if defined(__riscv)
4  /* In the project framework, corelib provides startup/linker and an
5   * exit_value section. */
6  #include "corelib.h"
7
8  /* Put result into the framework-defined section so testbench can read it
9   * from DTCM (0x4000). */
10
11  __attribute__((section(".exit_value")))
12  volatile uint32_t exit_value;
13
14  #else
15  /* On Windows/host, use stdio for printing */
16  #include <stdio.h>
17  #endif
18
19  static inline uint64_t rotr16(uint64_t x, unsigned r) {
20      r &= 63u;
21      return (x << r) | (x >> ((64u - r) & 63u));
22  }
23
24  static uint32_t signature_test(void) {
25      /* Static buffers: avoid huge stack on any platform */

```

```

23  static uint8_t  b8[256];
24  static uint16_t b16[128];
25  static uint32_t b32[128];
26  static uint64_t b64[128];
27
28  /* Init: stores */
29  for (uint32_t i = 0; i < 256; i++) b8[i]  = (uint8_t)(i * 3u + 1u);
30  for (uint32_t i = 0; i < 128; i++) b16[i] = (uint16_t)(i * 5u + 7u);
31  for (uint32_t i = 0; i < 128; i++) b32[i] = (uint32_t)(0x9e3779b9u ^ i)
    ;
32  for (uint32_t i = 0; i < 128; i++) b64[i] = 0x0123456789abcdefULL ^ (
    uint64_t)i;
33
34  uint64_t acc = 0x6a09e667f3bcc909ULL;
35  uint64_t a   = 0xdeadbeefcafebabeULL;
36  uint64_t b   = 0x0123456789abcdefULL;
37
38  for (uint32_t i = 0; i < 128; i++) {
39      /* loads (multi-width) */
40      uint64_t x8  = (uint64_t)(uint8_t)(b8[i] ^ b8[i + 1]);
41      uint64_t x16 = (uint64_t)(uint16_t)(b16[i] + (uint16_t)(b8[i] << 8)
    );
42      uint64_t x32 = (uint64_t)(uint32_t)(b32[i] ^ (uint32_t)(x16 * 13u))
    ;
43      uint64_t x64 = b64[i] + (x32 << (i & 31u));
44
45      /* ALU + shifts */
46      a  = (a + x64) ^ rotr16(b, i);
47      b  = (b - x32) + (a | (x16 << 17));
48      acc ^= (a & b) + (x8 | (x64 >> 3));
49
50      /* compares + branches */
51      if ((int64_t)a < (int64_t)b) acc += 0x1111u; else acc ^= 0x2222u;
52      if (a < b)                    acc += 0x33u;   else acc ^= 0x55u;
53
54      /* RV64M-ish ops (mul/div/rem) */
55      uint64_t den1 = (x32 | 3ULL) + 1ULL;
56      uint64_t den2 = (x16 | 5ULL) + 1ULL;
57      uint64_t m1   = (a ^ x64) * (b | 1ULL);
58      uint64_t d1   = m1 / den1;
59      uint64_t r1   = m1 % den2;
60      acc ^= (d1 + r1);
61
62      /* Signed div/rem */
63      int64_t sa = (int64_t)(a | 1ULL);
64      int64_t sb = (int64_t)(b | 3ULL);
65      acc ^= (uint64_t)(sa / sb);
66      acc ^= (uint64_t)(sa % sb);

```

```

67
68     /* 32-bit arithmetic (encourage W-path) */
69     int32_t  w0  = (int32_t)(acc ^ (acc >> 32));
70     int32_t  w1  = (int32_t)((uint32_t)x32 + i);
71     int32_t  w2  = (w0 + w1) ^ (w0 - w1);
72     int32_t  w3  = (w2 >> (i & 31u)) ^ (w2 << (i & 31u));
73     uint32_t uw2 = (uint32_t)w2;
74     uint32_t uw3 = (uw2 >> (i & 31u)) ^ (uw2 << (i & 31u));
75
76     int32_t  dw  = w3 / (w1 | 1);
77     int32_t  rw  = w3 % (w1 | 3);
78     uint32_t duw = uw3 / (uw2 | 1u);
79     uint32_t ruw = uw3 % (uw2 | 3u);
80
81     acc ^= (uint64_t)(uint32_t)dw ^ ((uint64_t)(uint32_t)rw << 7);
82     acc ^= (uint64_t)duw ^ ((uint64_t)ruw << 9);
83
84     /* switch (more control flow) */
85     switch (i & 3u) {
86         case 0: acc += 0x11111111ULL; break;
87         case 1: acc ^= 0x22222222ULL; break;
88         case 2: acc -= 0x33333333ULL; break;
89         default: acc = rotr164(acc, 7); break;
90     }
91
92     /* more stores */
93     b64[i] ^= acc;
94     b32[i] += (uint32_t)acc;
95 }
96
97 return (uint32_t)(acc ^ (acc >> 32));
98 }
99
100 int main(void) {
101     uint32_t sig = signature_test();
102
103     #if defined(__riscv)
104         /* Framework verification: testbench reads this from .exit_value / DTCM */
105         exit_value = sig;
106     #else
107         /* Windows/host: print signature */
108         printf("signature=0x%08x\n", sig);
109     #endif
110
111     return (int)sig;
112     //signature=0x857319ea
113 }

```

The following program is `fft_test.c`.

```

1  #include <stdint.h>
2
3  #if defined(__riscv)
4  #include "corelib.h"
5  __attribute__((section(".exit_value")))
6  volatile uint32_t exit_value;
7  #else
8  #include <stdio.h>
9  #endif
10
11 #define FFT_N 64
12 #define FFT_LOGN 6
13
14 static inline int32_t q15_mul(int32_t a, int32_t b) {
15     return (int32_t)((((int64_t)a * (int64_t)b) >> 15);
16 }
17
18 static inline uint32_t bit_reverse6(uint32_t x) {
19     uint32_t r = 0;
20     r |= ((x >> 0) & 1u) << 5;
21     r |= ((x >> 1) & 1u) << 4;
22     r |= ((x >> 2) & 1u) << 3;
23     r |= ((x >> 3) & 1u) << 2;
24     r |= ((x >> 4) & 1u) << 1;
25     r |= ((x >> 5) & 1u) << 0;
26     return r;
27 }
28
29 static void fft_compute(int32_t xr[FFT_N], int32_t xi[FFT_N]) {
30     static const int16_t wr[FFT_N / 2] = {
31         32767, 32610, 32138, 31357, 30273, 28899, 27245, 25330,
32         23170, 20787, 18204, 15446, 12539, 9512, 6393, 3212,
33         0, -3212, -6393, -9512, -12539, -15446, -18204, -20787,
34         -23170, -25330, -27245, -28899, -30273, -31357, -32138, -32610
35     };
36
37     static const int16_t wi[FFT_N / 2] = {
38         0, -3212, -6393, -9512, -12539, -15446, -18204, -20787,
39         -23170, -25330, -27245, -28899, -30273, -31357, -32138, -32610,
40         -32768, -32610, -32138, -31357, -30273, -28899, -27245, -25330,
41         -23170, -20787, -18204, -15446, -12539, -9512, -6393, -3212
42     };
43 }

```

```

44     for (uint32_t i = 0; i < FFT_N; i++) {
45         uint32_t j = bit_reverse6(i);
46         if (j > i) {
47             int32_t tr = xr[i], ti = xi[i];
48             xr[i] = xr[j]; xi[i] = xi[j];
49             xr[j] = tr;    xi[j] = ti;
50         }
51     }
52
53     for (uint32_t stage = 1; stage <= FFT_LOGN; stage++) {
54         uint32_t m = 1u << stage;
55         uint32_t mh = m >> 1;
56         uint32_t step = FFT_N / m;
57
58         for (uint32_t k = 0; k < FFT_N; k += m) {
59             for (uint32_t j = 0; j < mh; j++) {
60                 uint32_t tw = j * step;
61                 int32_t c = wr[tw];
62                 int32_t s = wi[tw];
63
64                 uint32_t i0 = k + j;
65                 uint32_t i1 = i0 + mh;
66
67                 int32_t ar = xr[i0], ai = xi[i0];
68                 int32_t br = xr[i1], bi = xi[i1];
69
70                 int32_t tr = q15_mul(br, c) - q15_mul(bi, s);
71                 int32_t ti = q15_mul(br, s) + q15_mul(bi, c);
72
73                 xr[i0] = ar + tr;
74                 xi[i0] = ai + ti;
75                 xr[i1] = ar - tr;
76                 xi[i1] = ai - ti;
77             }
78         }
79     }
80 }
81
82 static uint32_t signature_test(void) {
83     static int32_t xr[FFT_N];
84     static int32_t xi[FFT_N];
85
86     for (uint32_t i = 0; i < FFT_N; i++) {
87         xr[i] = (int32_t)((i * 97u + 31u) & 0x3ffu) - 512) << 5;
88         xi[i] = (int32_t)((i * 29u + 11u) & 0x1ffu) - 256) << 5;
89     }
90
91     fft_compute(xr, xi);

```



```

92
93 #if !defined(__riscv)
94     printf("FFT output (Q15-like fixed-point):\n");
95     for (uint32_t i = 0; i < FFT_N; i++) {
96         printf("X[%2u] = %d + j%d\n", i, xr[i], xi[i]);
97     }
98 #endif
99
100     uint64_t acc = 0x6a09e667f3bcc909ULL;
101     for (uint32_t i = 0; i < FFT_N; i++) {
102         acc ^= (uint64_t)(uint32_t)xr[i];
103         acc += ((uint64_t)(uint32_t)xi[i] << (i & 15u));
104         if (xr[i] < xi[i]) acc ^= 0x1111u;
105         else             acc += 0x2222u;
106
107         switch (i & 3u) {
108             case 0: acc += 0x11111111ULL; break;
109             case 1: acc ^= 0x22222222ULL; break;
110             case 2: acc -= 0x33333333ULL; break;
111             default: acc ^= (acc >> 7);   break;
112         }
113     }
114
115     return (uint32_t)(acc ^ (acc >> 32));
116 }
117
118 int main(void) {
119     uint32_t sig = signature_test();
120
121     #if defined(__riscv)
122         exit_value = sig;
123     else
124         printf("fft_signature=0x%08x\n", sig);
125         //fft_signature=0x2de8aac0
126     #endif
127
128     return (int)sig;
129 }

```

The following program is aes128_test.c for validation.

```

1 #include <stdint.h>
2
3 #if defined(__riscv)
4 #include "corelib.h"
5 __attribute__((section(".exit_value")))
6 volatile uint32_t exit_value;

```

```

7  #else
8  #include <stdio.h>
9  #endif
10
11 static const uint8_t sbox[256] = {
12     0x63,0x7c,0x77,0x7b,0xf2,0x6b,0x6f,0xc5,0x30,0x01,0x67,0x2b,0xfe,0xd7,0
    xab,0x76,
13     0xca,0x82,0xc9,0x7d,0xfa,0x59,0x47,0xf0,0xad,0xd4,0xa2,0xaf,0x9c,0xa4,0
    x72,0xc0,
14     0xb7,0xfd,0x93,0x26,0x36,0x3f,0xf7,0xcc,0x34,0xa5,0xe5,0xf1,0x71,0xd8,0
    x31,0x15,
15     0x04,0xc7,0x23,0xc3,0x18,0x96,0x05,0x9a,0x07,0x12,0x80,0xe2,0xeb,0x27,0
    xb2,0x75,
16     0x09,0x83,0x2c,0x1a,0x1b,0x6e,0x5a,0xa0,0x52,0x3b,0xd6,0xb3,0x29,0xe3,0
    x2f,0x84,
17     0x53,0xd1,0x00,0xed,0x20,0xfc,0xb1,0x5b,0x6a,0xcb,0xbe,0x39,0x4a,0x4c,0
    x58,0xcf,
18     0xd0,0xef,0xaa,0xfb,0x43,0x4d,0x33,0x85,0x45,0xf9,0x02,0x7f,0x50,0x3c,0
    x9f,0xa8,
19     0x51,0xa3,0x40,0x8f,0x92,0x9d,0x38,0xf5,0xbc,0xb6,0xda,0x21,0x10,0xff,0
    xf3,0xd2,
20     0xcd,0x0c,0x13,0xec,0x5f,0x97,0x44,0x17,0xc4,0xa7,0x7e,0x3d,0x64,0x5d,0
    x19,0x73,
21     0x60,0x81,0x4f,0xdc,0x22,0x2a,0x90,0x88,0x46,0xee,0xb8,0x14,0xde,0x5e,0
    x0b,0xdb,
22     0xe0,0x32,0x3a,0x0a,0x49,0x06,0x24,0x5c,0xc2,0xd3,0xac,0x62,0x91,0x95,0
    xe4,0x79,
23     0xe7,0xc8,0x37,0x6d,0x8d,0xd5,0x4e,0xa9,0x6c,0x56,0xf4,0xea,0x65,0x7a,0
    xae,0x08,
24     0xba,0x78,0x25,0x2e,0x1c,0xa6,0xb4,0xc6,0xe8,0xdd,0x74,0x1f,0x4b,0xbd,0
    x8b,0x8a,
25     0x70,0x3e,0xb5,0x66,0x48,0x03,0xf6,0x0e,0x61,0x35,0x57,0xb9,0x86,0xc1,0
    x1d,0x9e,
26     0xe1,0xf8,0x98,0x11,0x69,0xd9,0x8e,0x94,0x9b,0x1e,0x87,0xe9,0xce,0x55,0
    x28,0xdf,
27     0x8c,0xa1,0x89,0x0d,0xbf,0xe6,0x42,0x68,0x41,0x99,0x2d,0x0f,0xb0,0x54,0
    xbb,0x16
28 };
29
30 static const uint8_t rcon[10] = {0x01,0x02,0x04,0x08,0x10,0x20,0x40,0x80,0
    x1B,0x36};
31
32 static inline uint8_t xtime(uint8_t x) { return (uint8_t)((x << 1) ^ ((x &
    0x80u) ? 0x1bu : 0)); }
33 static inline uint8_t mul2(uint8_t x) { return xtime(x); }
34 static inline uint8_t mul3(uint8_t x) { return (uint8_t)(xtime(x) ^ x); }
35
36 static void sub_bytes(uint8_t st[16]) { for (uint32_t i = 0; i < 16; i++)

```

```

    st[i] = sbbox[st[i]]; }
37
38 static void shift_rows(uint8_t st[16]) {
39     uint8_t t;
40     t = st[1]; st[1] = st[5]; st[5] = st[9]; st[9] = st[13]; st[13] =
        t;
41     t = st[2]; st[2] = st[10]; st[10] = t;
42     t = st[6]; st[6] = st[14]; st[14] = t;
43     t = st[15]; st[15] = st[11]; st[11] = st[7]; st[7] = st[3]; st[3] =
        t;
44 }
45
46 static void mix_columns(uint8_t st[16]) {
47     for (uint32_t c = 0; c < 4; c++) {
48         uint32_t i = c * 4;
49         uint8_t a0 = st[i+0], a1 = st[i+1], a2 = st[i+2], a3 = st[i+3];
50         st[i+0] = (uint8_t)(mul2(a0) ^ mul3(a1) ^ a2 ^ a3);
51         st[i+1] = (uint8_t)(a0 ^ mul2(a1) ^ mul3(a2) ^ a3);
52         st[i+2] = (uint8_t)(a0 ^ a1 ^ mul2(a2) ^ mul3(a3));
53         st[i+3] = (uint8_t)(mul3(a0) ^ a1 ^ a2 ^ mul2(a3));
54     }
55 }
56
57 static void add_round_key(uint8_t st[16], const uint8_t *rk) {
58     for (uint32_t i = 0; i < 16; i++) st[i] ^= rk[i];
59 }
60
61 static void key_expansion(const uint8_t key[16], uint8_t round_keys[176]) {
62     for (uint32_t i = 0; i < 16; i++) round_keys[i] = key[i];
63
64     uint32_t bytes_generated = 16, rcon_idx = 0;
65     uint8_t temp[4];
66
67     while (bytes_generated < 176) {
68         for (uint32_t i = 0; i < 4; i++) temp[i] = round_keys[
            bytes_generated - 4 + i];
69
70         if ((bytes_generated & 15u) == 0) {
71             uint8_t t = temp[0];
72             temp[0] = sbbox[temp[1]] ^ rcon[rcon_idx++];
73             temp[1] = sbbox[temp[2]];
74             temp[2] = sbbox[temp[3]];
75             temp[3] = sbbox[t];
76         }
77
78         for (uint32_t i = 0; i < 4; i++) {
79             round_keys[bytes_generated] = round_keys[bytes_generated - 16]
                ^ temp[i];

```

```

80         bytes_generated++;
81     }
82 }
83 }
84
85 static void aes128_encrypt_block(uint8_t st[16], const uint8_t round_keys
[176]) {
86     add_round_key(st, &round_keys[0]);
87     for (uint32_t round = 1; round <= 9; round++) {
88         sub_bytes(st);
89         shift_rows(st);
90         mix_columns(st);
91         add_round_key(st, &round_keys[round * 16]);
92     }
93     sub_bytes(st);
94     shift_rows(st);
95     add_round_key(st, &round_keys[160]);
96 }
97
98 static uint32_t signature_test(void) {
99     static uint8_t key[16], pt[16], st[16], round_keys[176];
100
101     for (uint32_t i = 0; i < 16; i++) {
102         key[i] = (uint8_t)(i * 17u + 3u);
103         pt[i] = (uint8_t)(i * 29u + 11u);
104         st[i] = pt[i];
105     }
106
107     key_expansion(key, round_keys);
108     aes128_encrypt_block(st, round_keys);
109
110 #if !defined(__riscv)
111     printf("AES-128 ciphertext:\n");
112     for (uint32_t i = 0; i < 16; i++) {
113         printf("%02x", st[i]);
114         if ((i & 3u) == 3u) printf(" ");
115     }
116     printf("\n");
117 #endif
118
119     uint64_t acc = 0x510e527fade682d1ULL;
120     for (uint32_t i = 0; i < 176; i++) {
121         acc ^= (uint64_t)round_keys[i];
122         acc += ((uint64_t)round_keys[i] << (i & 7u));
123     }
124     for (uint32_t i = 0; i < 16; i++) {
125         acc ^= (uint64_t)st[i];
126         if (st[i] & 1u) acc ^= 0x1111u;

```

```

127         else                acc += 0x2222u;
128
129         switch (i & 3u) {
130             case 0: acc += 0x11111111ULL; break;
131             case 1: acc ^= 0x22222222ULL; break;
132             case 2: acc -= 0x33333333ULL; break;
133             default: acc ^= (acc >> 5); break;
134         }
135     }
136
137     return (uint32_t)(acc ^ (acc >> 32));
138 }
139
140 int main(void) {
141     uint32_t sig = signature_test();
142
143     #if defined(__riscv)
144         exit_value = sig;
145     #else
146         printf("aes128_signature=0x%08x\n", sig);
147         //aes128_signature=0xc063dcde
148     #endif
149
150     return (int)sig;
151 }

```

The following program is `matmul_test.c` for validation.

```

1  #include <stdint.h>
2  #if defined(__riscv)
3  #include "corelib.h"
4  __attribute__((section(".exit_value")))
5  volatile uint32_t exit_value;
6  #else
7  #include <stdio.h>
8  #endif
9  static uint32_t signature_test(void) {
10     static int32_t A[16][16];
11     static int32_t B[16][16];
12     static int32_t C[16][16];
13     for (uint32_t i = 0; i < 16; i++) {
14         for (uint32_t j = 0; j < 16; j++) {
15             A[i][j] = (int32_t)((i * 17u + j * 7u + 3u) & 0xffu) - 128;
16             B[i][j] = (int32_t)((i * 11u + j * 13u + 5u) & 0xffu) - 128;
17             C[i][j] = 0;
18         }
19     }

```

```

20     for (uint32_t i = 0; i < 16; i++) {
21         for (uint32_t j = 0; j < 16; j++) {
22             int64_t sum = 0;
23             for (uint32_t k = 0; k < 16; k++) {
24                 sum += (int64_t)A[i][k] * (int64_t)B[k][j];
25             }
26             C[i][j] = (int32_t)sum;
27         }
28     }
29     #if !defined(__riscv)
30     printf("Matrix multiplication result C=A*B:\n");
31     for (uint32_t i = 0; i < 16; i++) {
32         for (uint32_t j = 0; j < 16; j++) {
33             printf("%8d ", C[i][j]);
34         }
35         printf("\n");
36     }
37     #endif
38     uint64_t acc = 0x510e527fade682d1ULL;
39     for (uint32_t i = 0; i < 16; i++) {
40         for (uint32_t j = 0; j < 16; j++) {
41             acc ^= (uint64_t)(uint32_t)C[i][j];
42             acc += ((uint64_t)(uint32_t)C[i][j] << ((i + j) & 15u));
43
44             if (C[i][j] < 0) acc ^= 0x1111u;
45             else acc += 0x2222u;
46
47             switch ((i + j) & 3u) {
48                 case 0: acc += 0x11111111ULL; break;
49                 case 1: acc ^= 0x22222222ULL; break;
50                 case 2: acc -= 0x33333333ULL; break;
51                 default: acc ^= (acc >> 11); break;
52             }
53         }
54     }
55     return (uint32_t)(acc ^ (acc >> 32));
56 }
57 int main(void) {
58     uint32_t sig = signature_test();
59     #if defined(__riscv)
60         exit_value = sig;
61     #else
62         printf("matmul_signature=0x%08x\n", sig);
63     #endif
64     return (int)sig;
65 }

```